

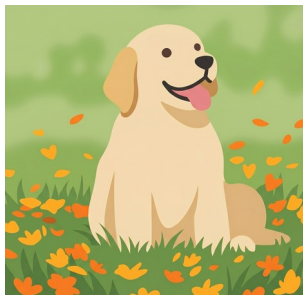
PhD Tutorial **Explainable Artificial Intelligence (xAI) in Chemistry**

Jonas Teufel · AIMAT Group
Institute of Nanotechnology · INT



Why should we use XAI in chemistry?

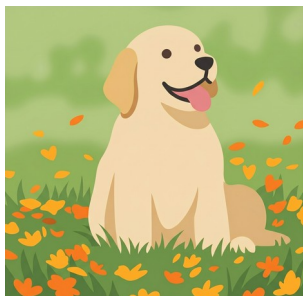
Does the image show a cat or a dog?



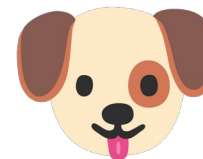
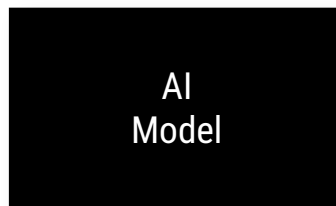
Input Image

Why should we use XAI in chemistry?

Does the image show a cat or a dog?

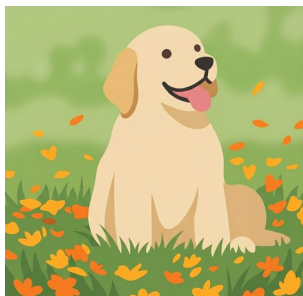


Input Image

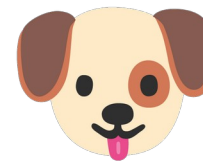


Why should we use XAI in chemistry?

Does the image show a cat or a dog?

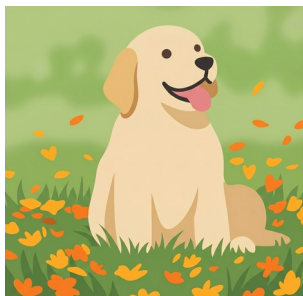


Input Image

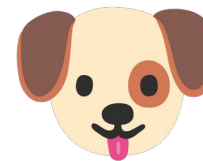


Why should we use XAI in chemistry?

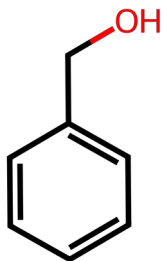
Does the image show a cat or a dog?



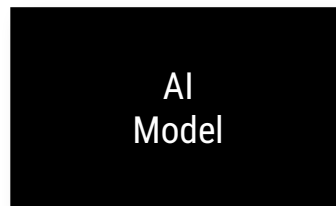
Input Image



How soluble is the molecule?

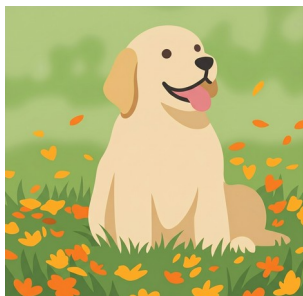


Input Molecule

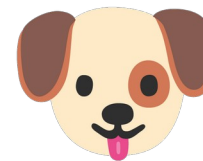


Why should we use XAI in chemistry?

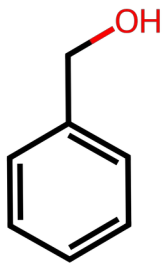
Does the image show a cat or a dog?



Input Image



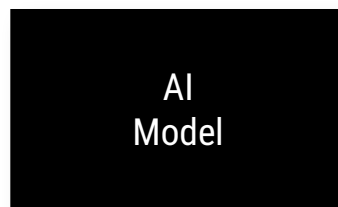
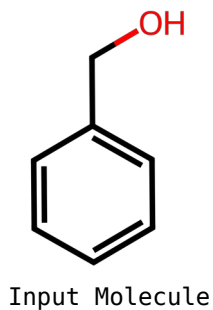
How soluble is the molecule?



Input Molecule

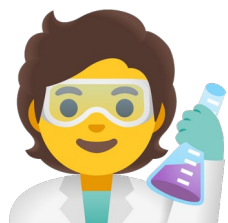


Can we gain understanding from complex models?



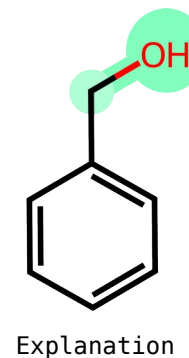
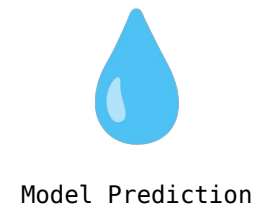
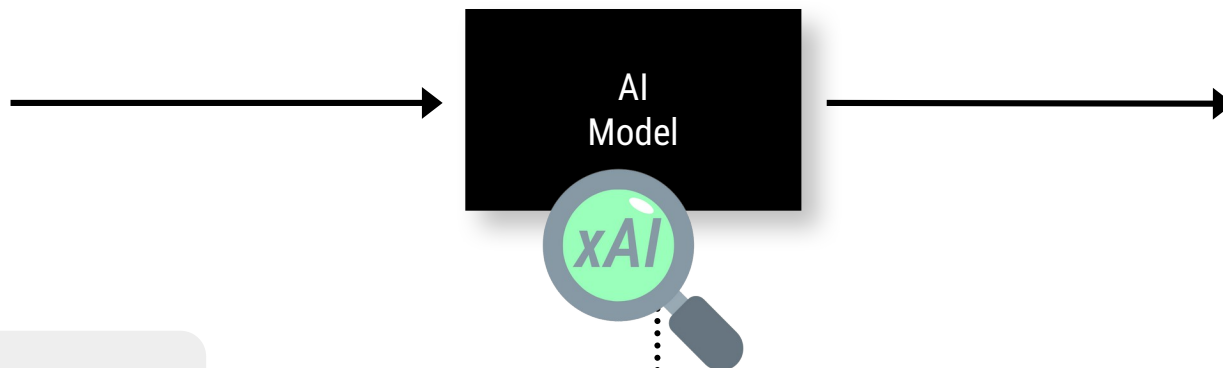
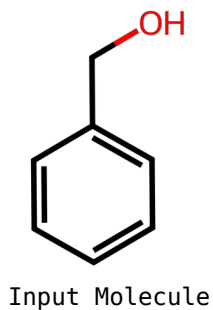
Model Prediction

Why does it do that?

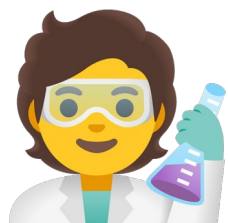


Chemist / Expert

Can we gain understanding from complex models?

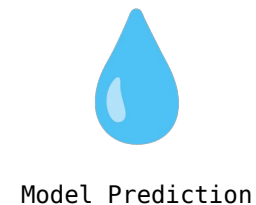
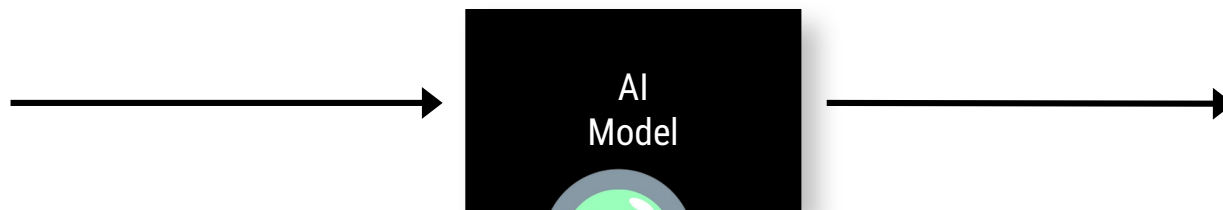
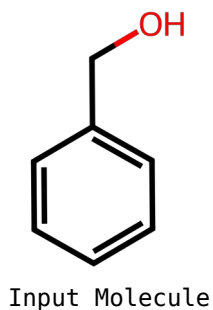


Why does it do that?

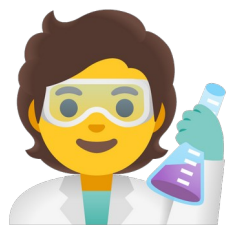


Chemist / Expert

Can we gain understanding from complex models?



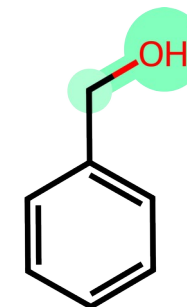
Why does it do that?



Chemist / Expert



This part is particularly important!



Explanation

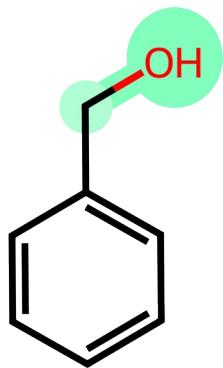
Talk Overview

Talk Overview

02

Attributional Explanations

Which parts are especially important?

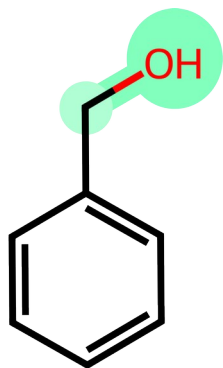


Talk Overview

02

Attributional Explanations

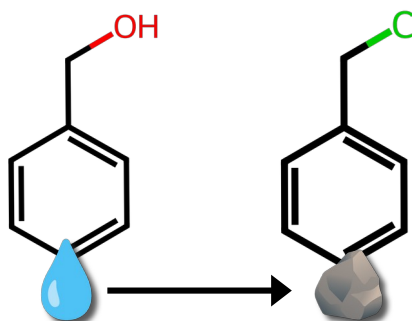
Which parts are especially important?



03

Counterfactuals

What would I have to change in the input to change the outcome?

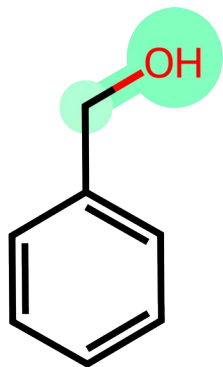


Talk Overview

02

Attributional Explanations

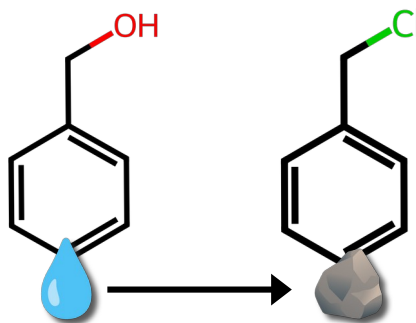
Which parts are especially important?



03

Counterfactuals

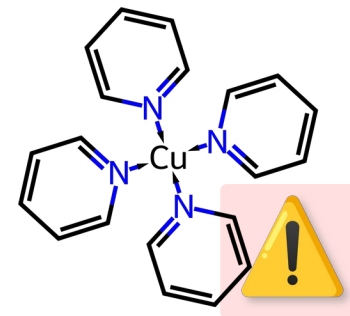
What would I have to change in the input to change the outcome?



04

Uncertainty Estimation

When can I trust the model predictions and when not?



GUIDE

✦ Companion Jupyter Notebook Tutorials

https://github.com/the16thpythonist/xai_chem_review.git

02

Attributional Explanations



03

Counterfactuals

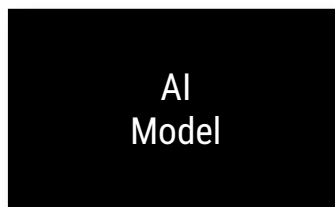
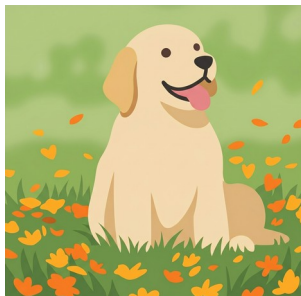


04

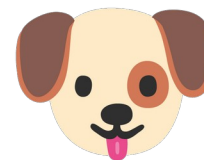
Uncertainty Estimation



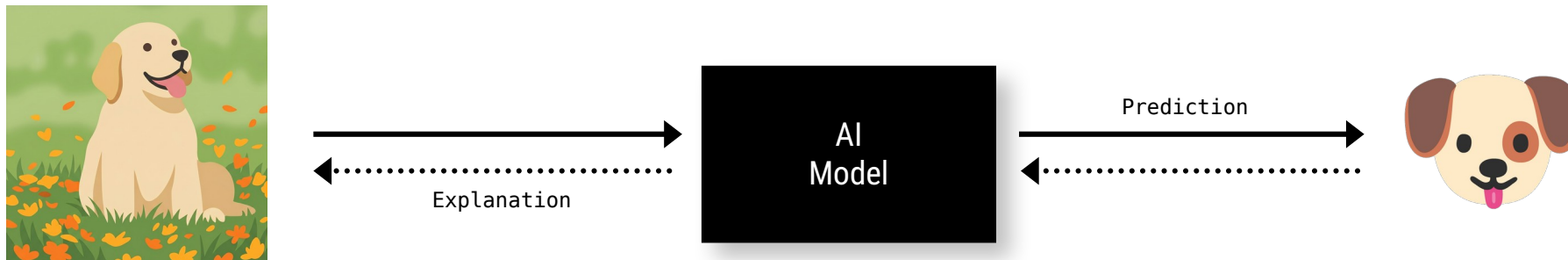
What are attributional explanations?



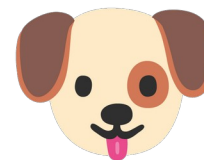
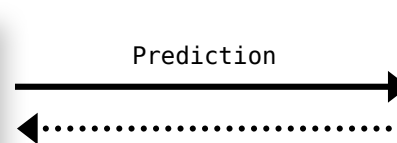
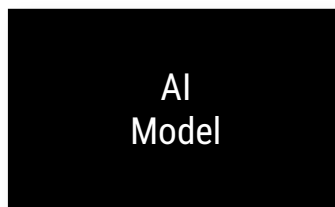
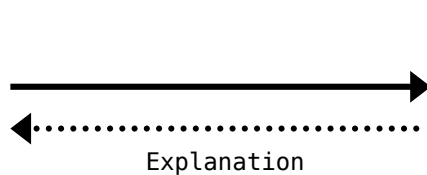
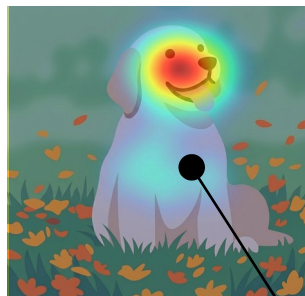
Prediction



What are attributional explanations?

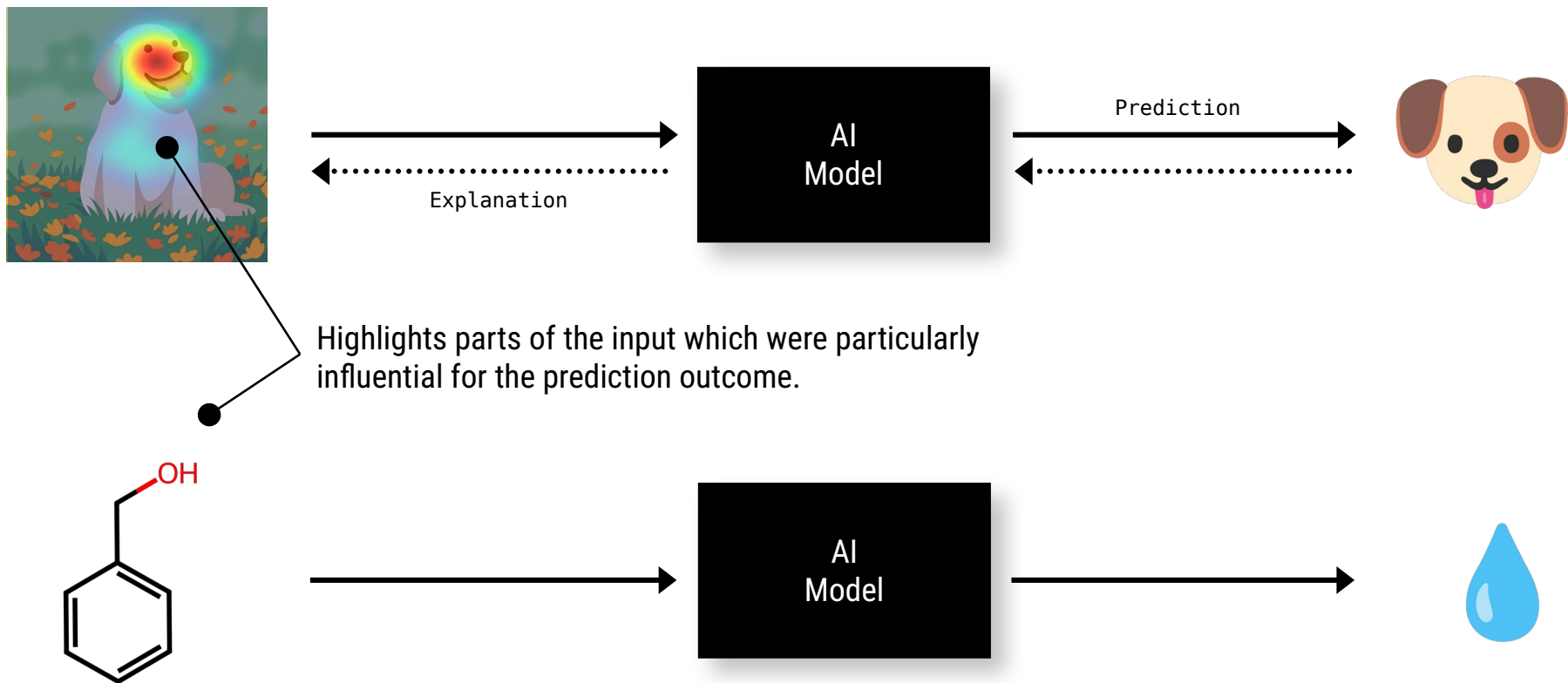


What are attributional explanations?

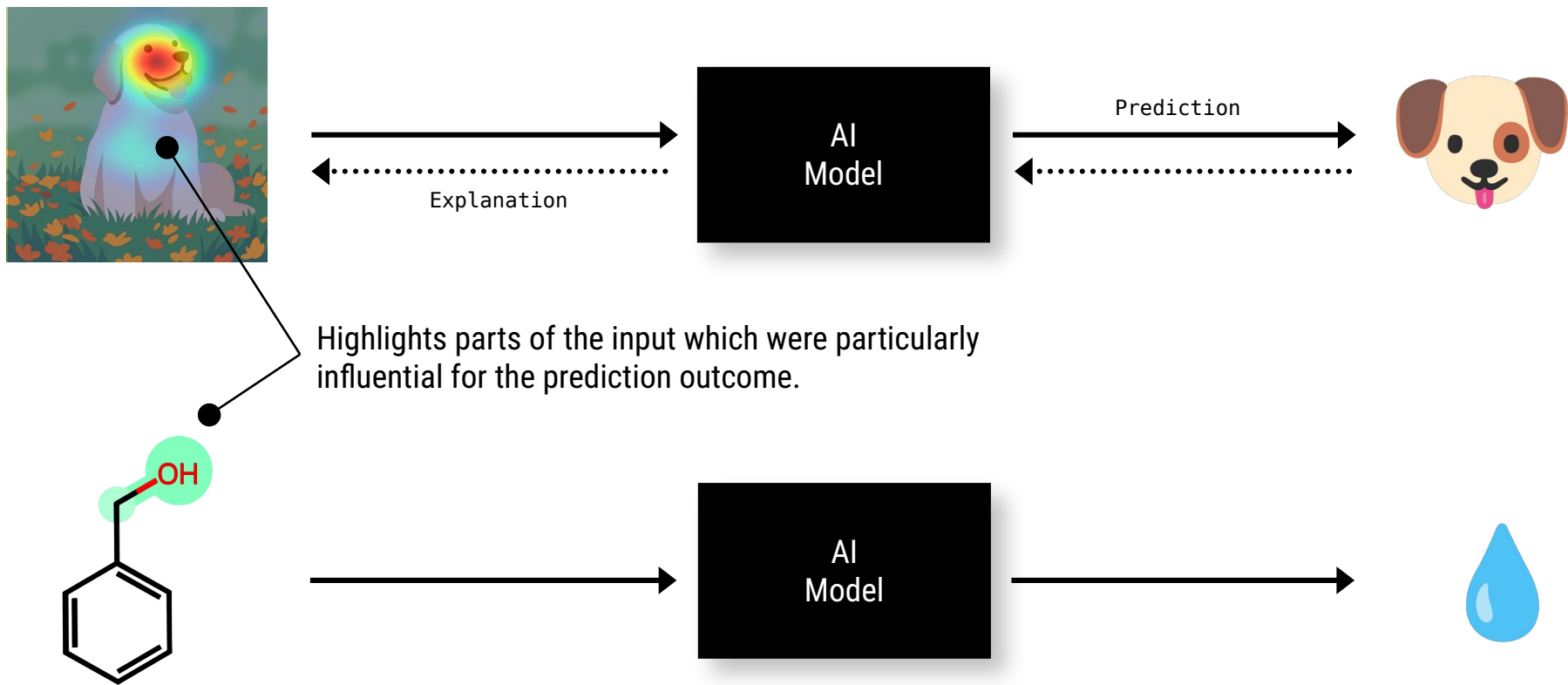


Highlights parts of the input which were particularly influential for the prediction outcome.

What are attributional explanations?



What are attributional explanations?



ZORRO: Valid, Sparse and Stable GNN Explanations

[1] Funke et al. *ZORRO: Valid, Sparse and Stable Explanations for Graph Neural Networks*. NIPS'17 [link](#)

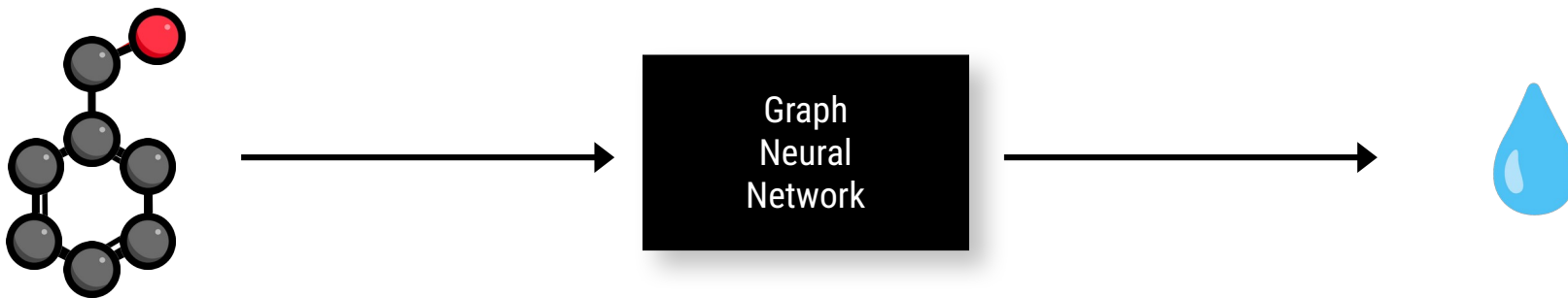
Goal: Which minimal subset of the graph leads to the approximately the same prediction?



ZORRO: Valid, Sparse and Stable GNN Explanations

[1] Funke et al. *ZORRO: Valid, Sparse and Stable Explanations for Graph Neural Networks*. NIPS'17 [link](#)

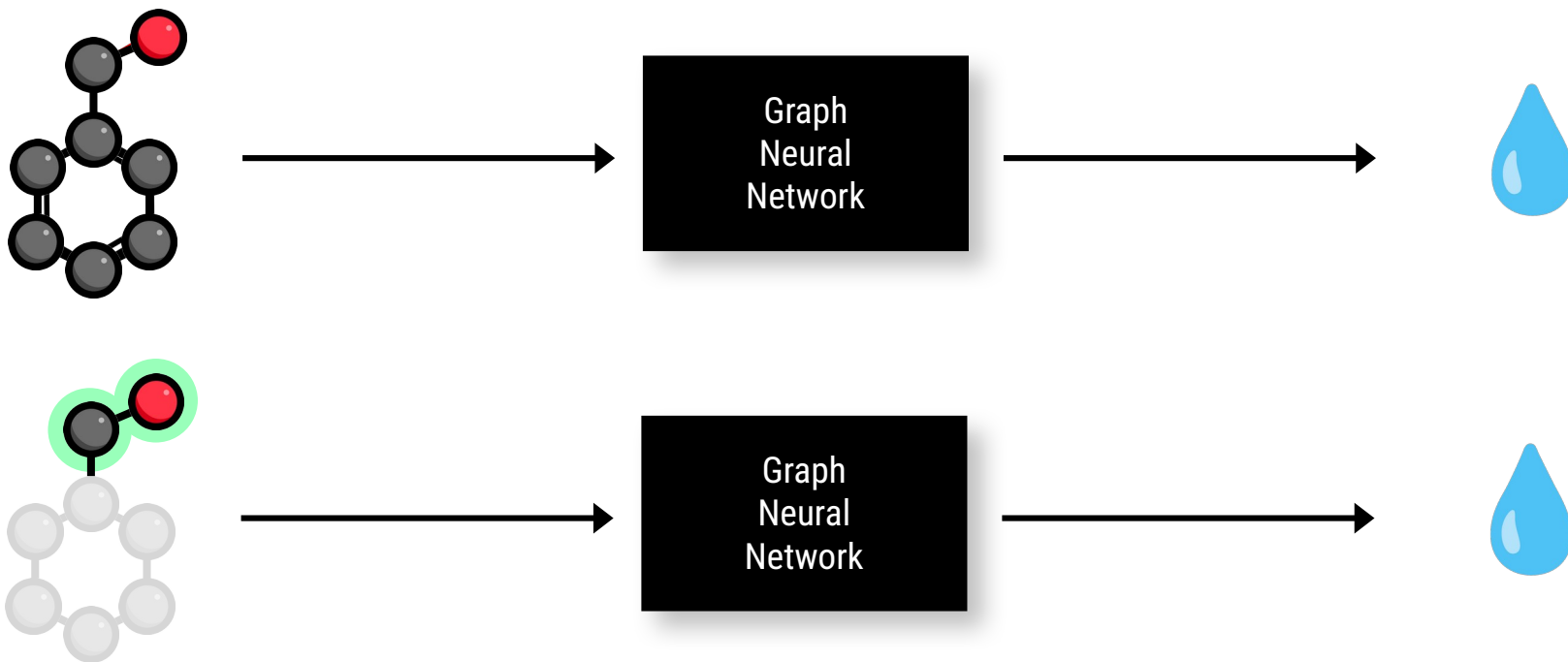
Goal: Which minimal subset of the graph leads to the approximately the same prediction?



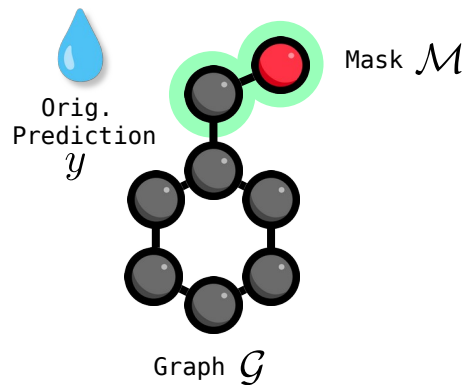
ZORRO: Valid, Sparse and Stable GNN Explanations

[1] Funke et al. *ZORRO: Valid, Sparse and Stable Explanations for Graph Neural Networks*. NIPS'17 [link](#)

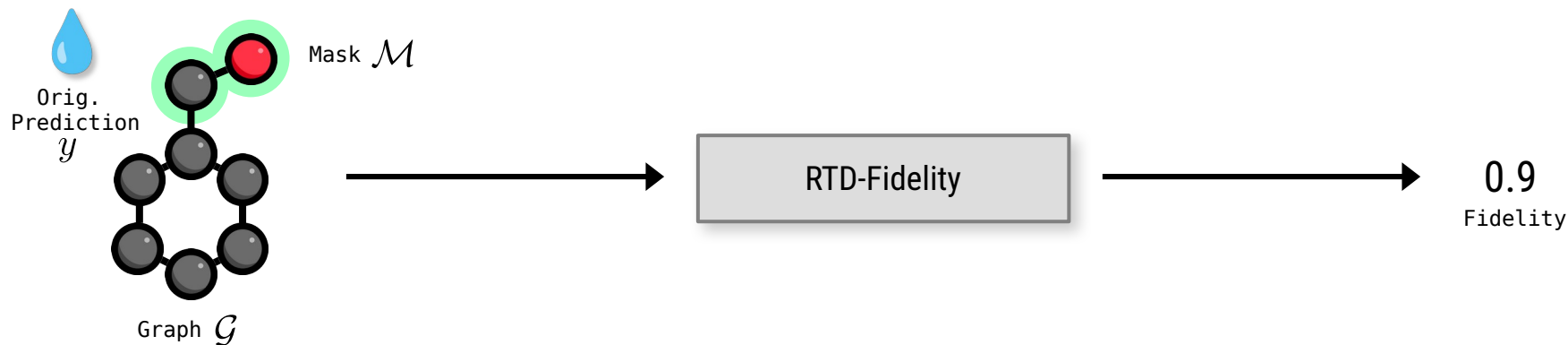
Goal: Which minimal subset of the graph leads to the approximately the same prediction?



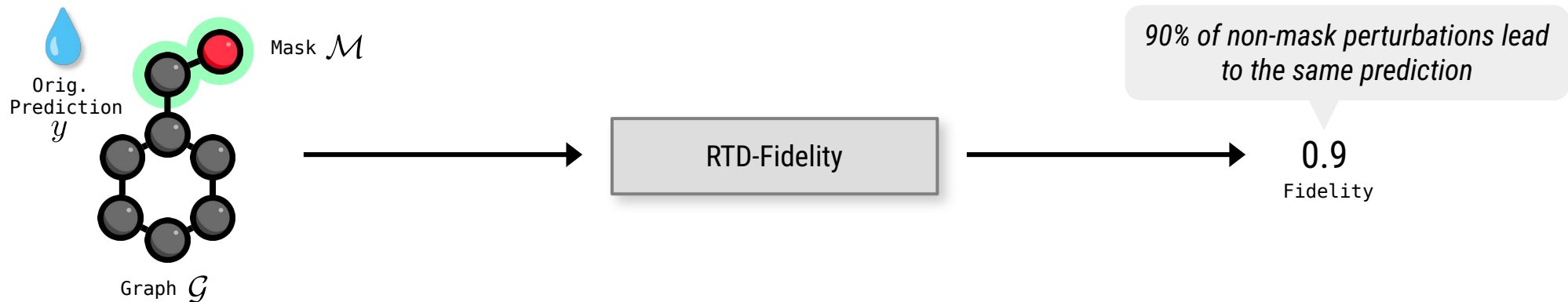
ZORRO: Valid, Sparse and Stable GNN Explanations II



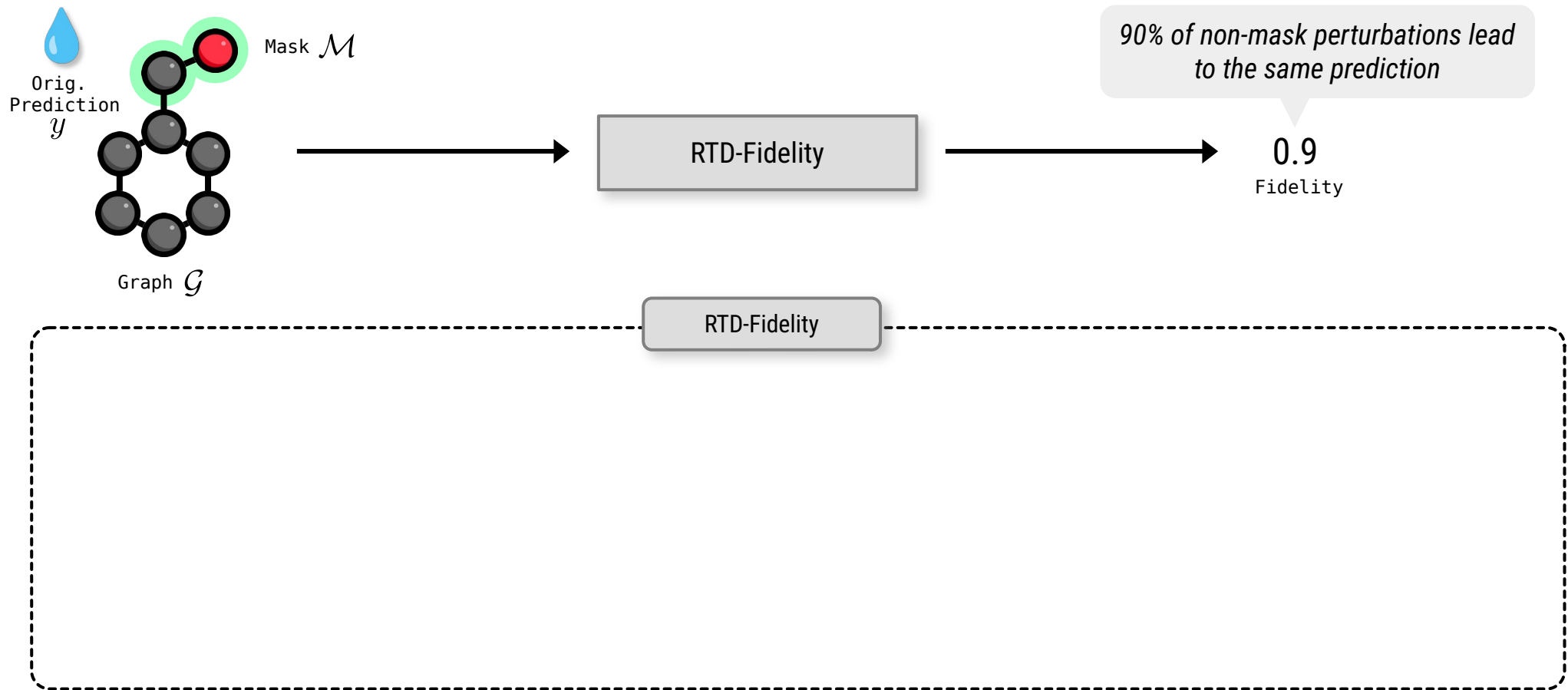
ZORRO: Valid, Sparse and Stable GNN Explanations II



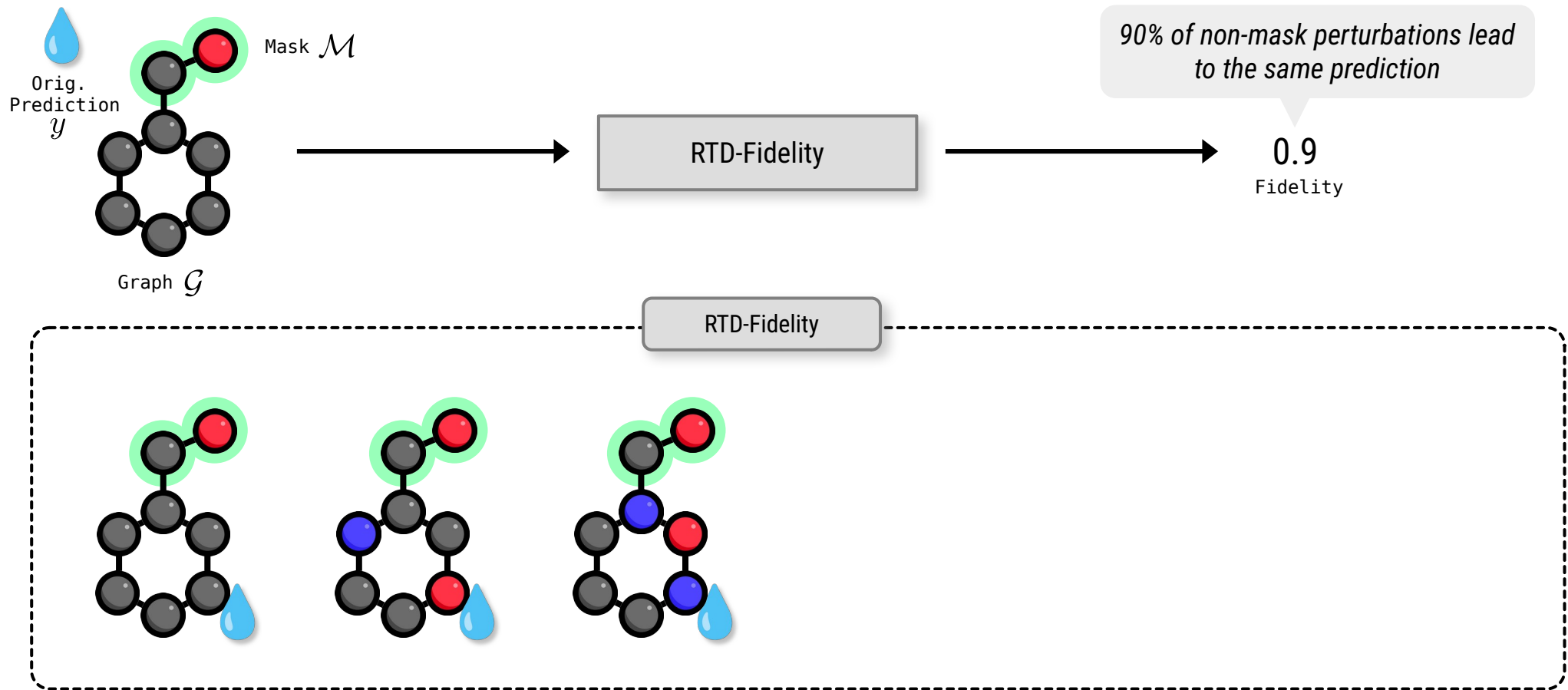
ZORRO: Valid, Sparse and Stable GNN Explanations II



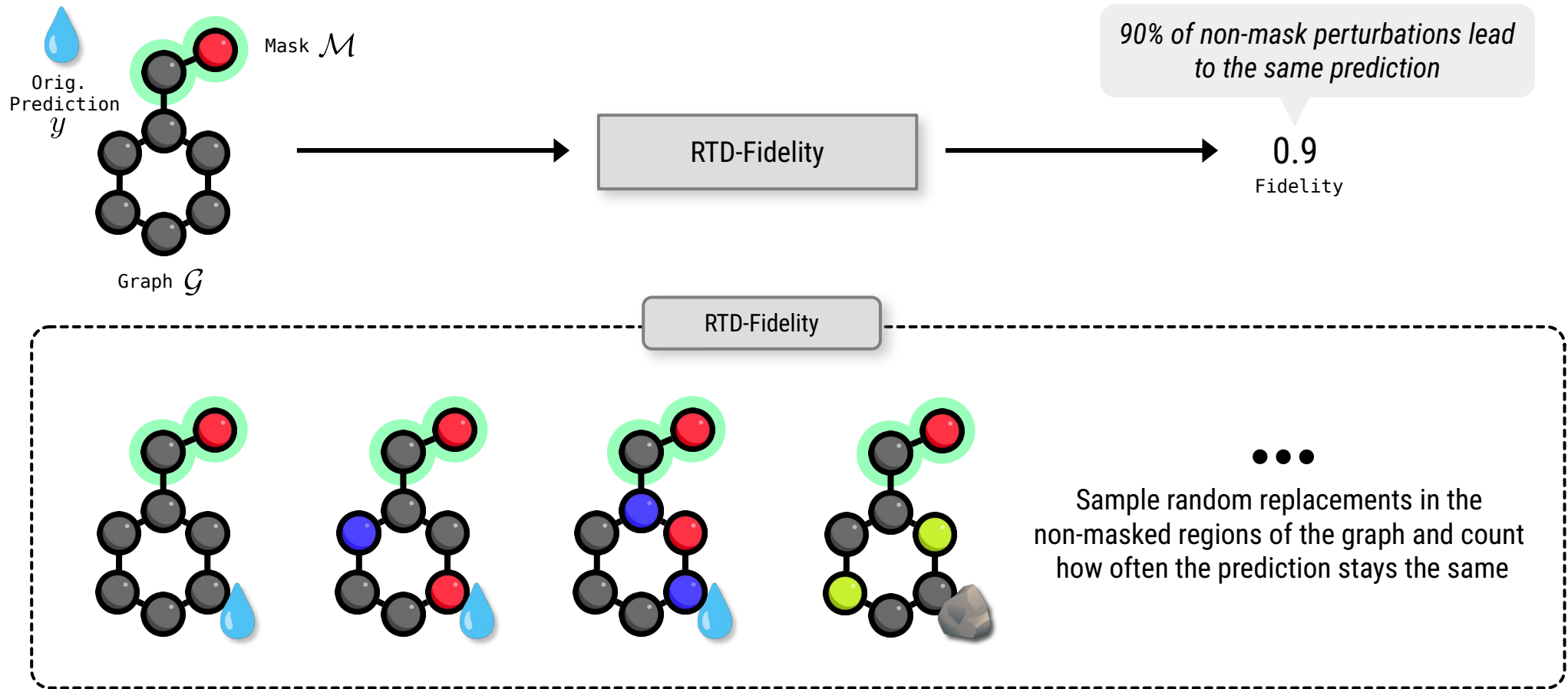
ZORRO: Valid, Sparse and Stable GNN Explanations II



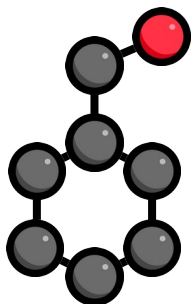
ZORRO: Valid, Sparse and Stable GNN Explanations II



ZORRO: Valid, Sparse and Stable GNN Explanations II



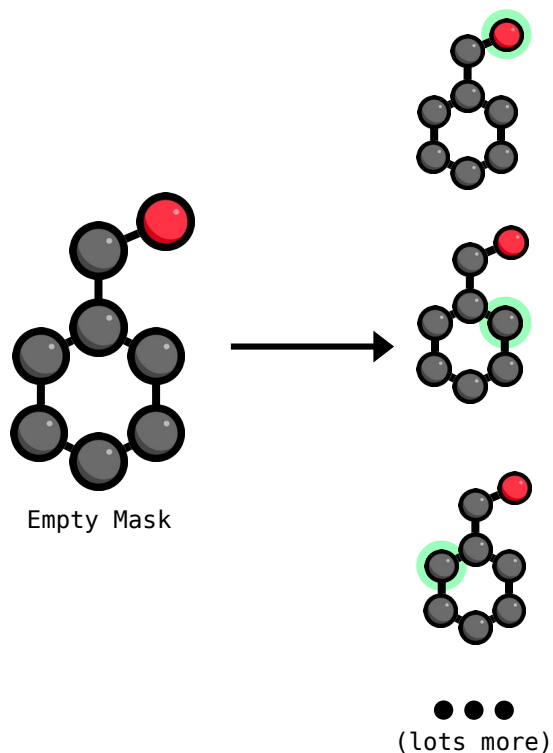
ZORRO: Valid, Sparse and Stable GNN Explanations III



Empty Mask

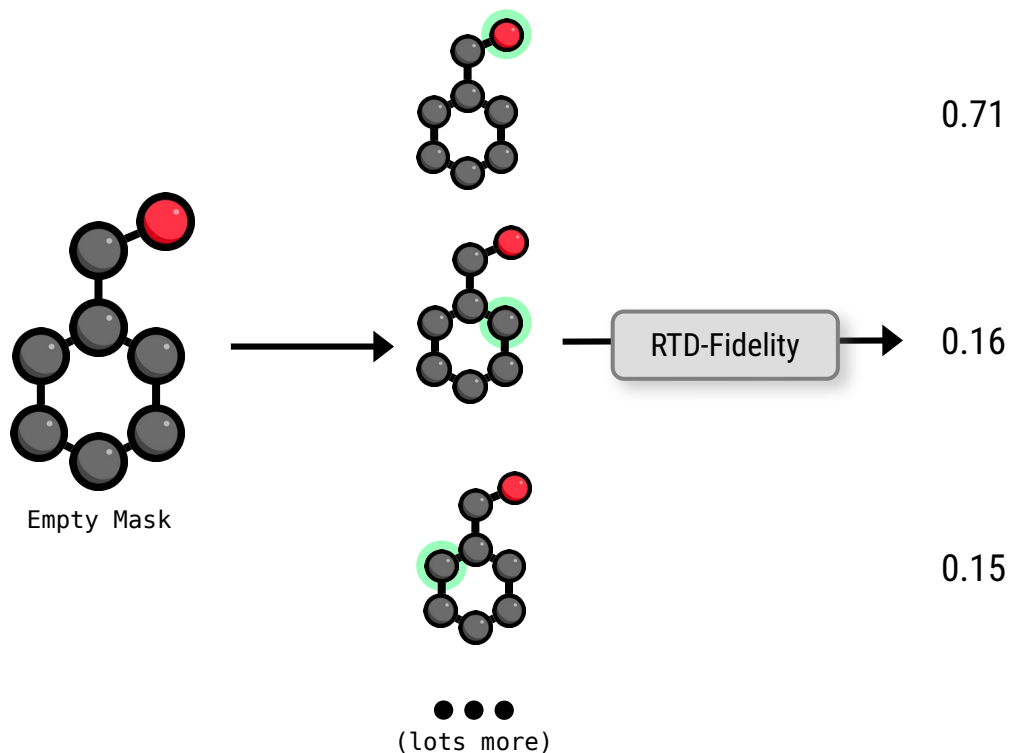
ZORRO: Valid, Sparse and Stable GNN Explanations III

I. Evaluate all possible expansions



ZORRO: Valid, Sparse and Stable GNN Explanations III

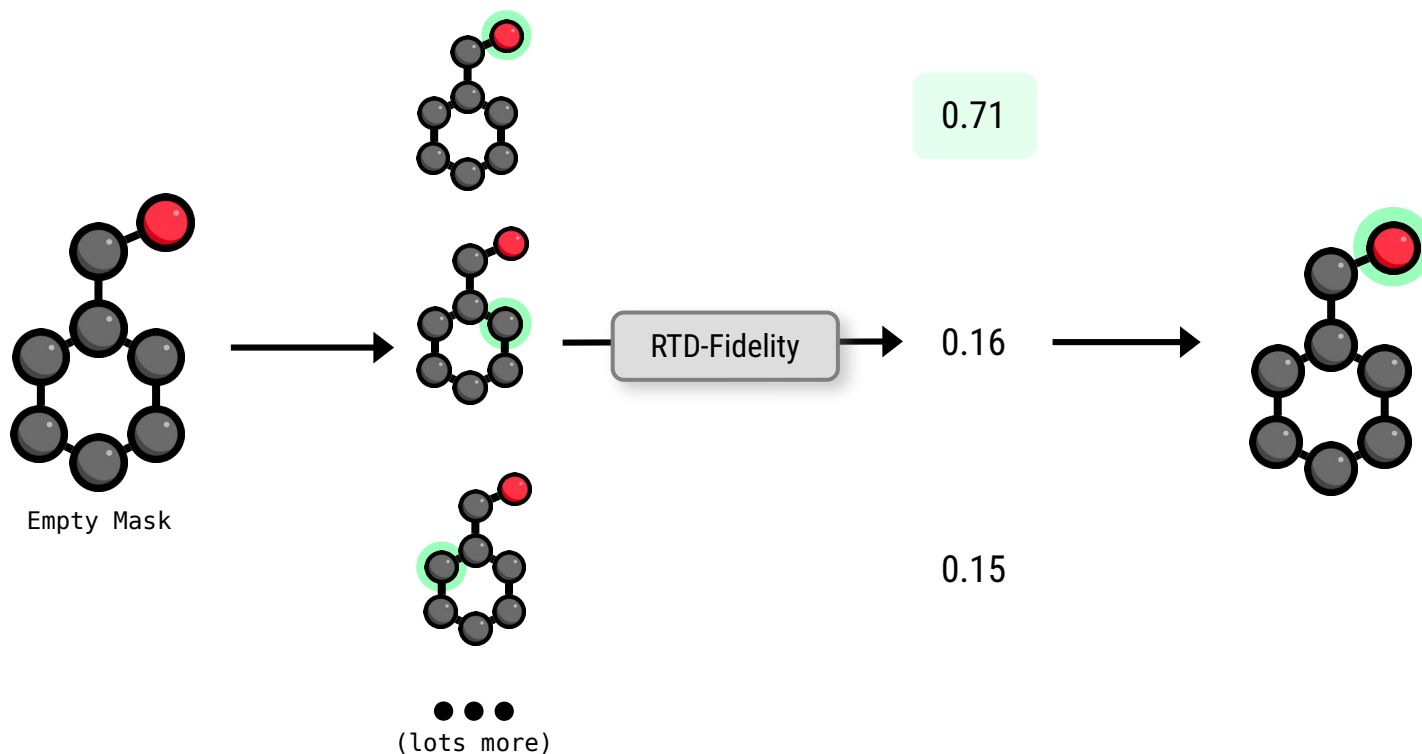
I. Evaluate all possible expansions



ZORRO: Valid, Sparse and Stable GNN Explanations III

I. Evaluate all possible expansions

II. Greedily pick best

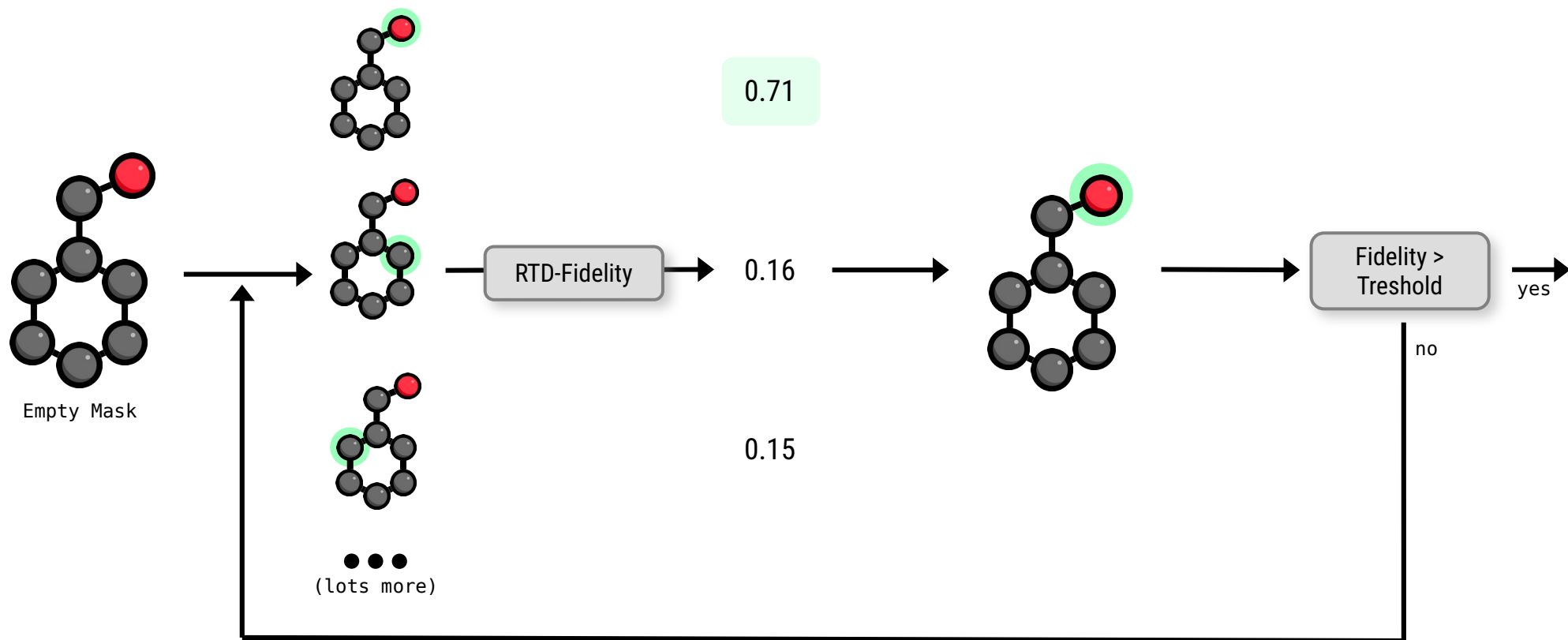


ZORRO: Valid, Sparse and Stable GNN Explanations III

I. Evaluate all possible expansions

II. Greedily pick best

III. Repeat



GUIDE

✦ ZORRO: Valid, Sparse and Stable GNN Explanations

```
import torch
from torch_geometric.utils.data import Data
from zorrito import Zorrito
```

```
# 1 - configuring the explainer object
```

```
# 2 - generating the explanation
```

GUIDE

✦ ZORRO: Valid, Sparse and Stable GNN Explanations

```
import torch
from torch_geometric.utils.data import Data
from zorrito import Zorro

# 1 - configuring the explainer object
explainer = Zorro(
    model=model,
    task="graph",
    select="nodes_only",
    noise_mode="row",
    noise_pool=torch.cat([g.x for g in dataset], dim=0),
    samples=100,
)

# 2 - generating the explanation
```

graph-level tasks vs. node-level tasks

*Select all nodes from the actual datasets
as valid noise substitutions for the
RTD-Fidelity calculation*

GUIDE

✦ ZORRO: Valid, Sparse and Stable GNN Explanations

```
import torch
from torch_geometric.utils.data import Data
from zorrito import Zorro

# 1 - configuring the explainer object
explainer = Zorro(
    model=model,
    task="graph",
    select="nodes_only",
    noise_mode="row",
    noise_pool=torch.cat([g.x for g in dataset], dim=0),
    samples=100,
)

# 2 - generating the explanation
SMILES: str = "C1=CC=CC=C1CO"
graph: Data = data_from_smiles(SMILES)
```

graph-level tasks vs. node-level tasks

*Select all nodes from the actual datasets
as valid noise substitutions for the
RTD-Fidelity calculation*

*Either break at threshold or if there is no
greedy improvement anymore*

GUIDE

✦ ZORRO: Valid, Sparse and Stable GNN Explanations

```
import torch
from torch_geometric.utils.data import Data
from zorrito import Zorro

# 1 - configuring the explainer object
explainer = Zorro(
    model=model,
    task="graph",
    select="nodes_only",
    noise_mode="row",
    noise_pool=torch.cat([g.x for g in dataset], dim=0),
    samples=100,
)

# 2 - generating the explanation
SMILES: str = "C1=CC=CC=C1CO"
graph: Data = data_from_smiles(SMILES)
result = explainer.explain(
    x=graph.x,
    edge_index=graph.edge_index,
    fidelity_threshold=0.85,
)
```

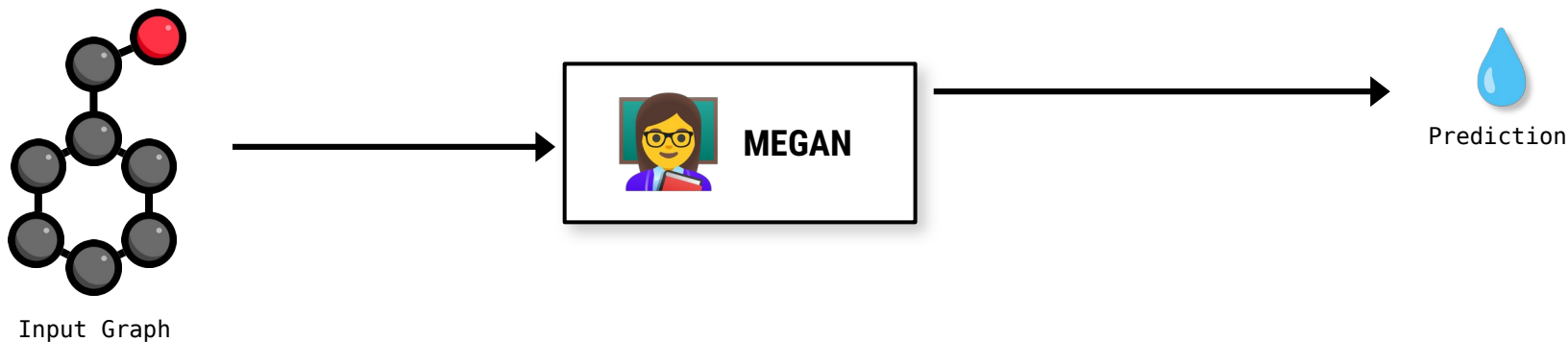
graph-level tasks vs. node-level tasks

*Select all nodes from the actual datasets
as valid noise substitutions for the
RTD-Fidelity calculation*

*Either break at threshold or if there is no
greedy improvement anymore*

Self-explaining graph neural networks

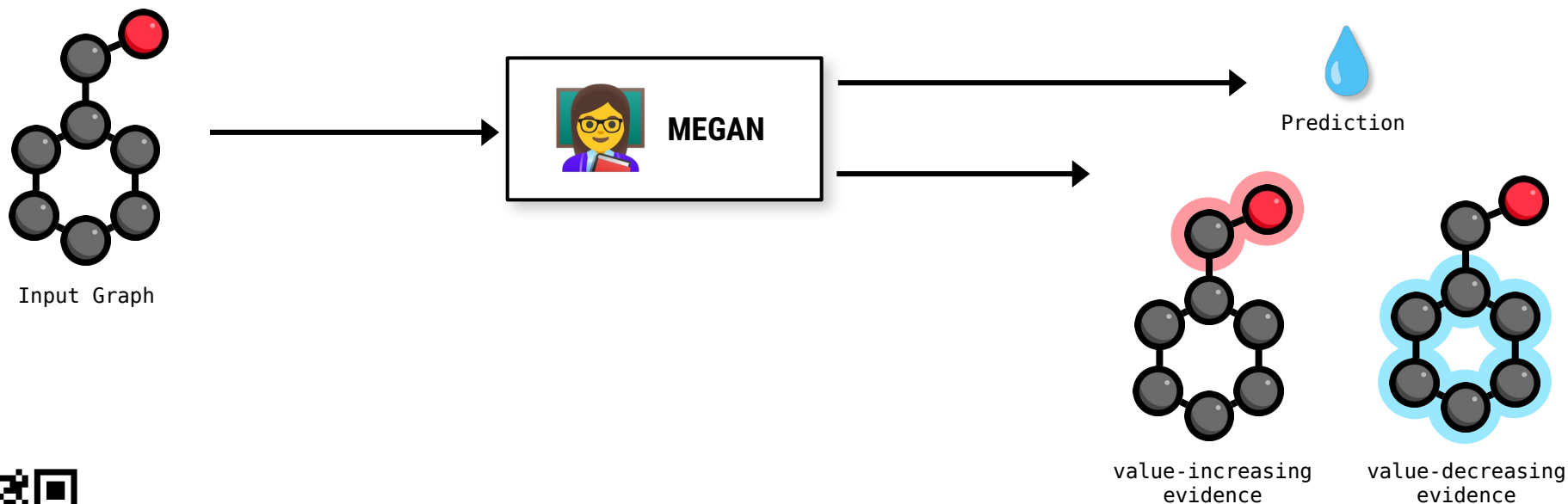
[2] Teufel et al. *MEGAN: Multi-Explanation Graph Attention Network*. XAI'22



Paper Online

Self-explaining graph neural networks

[2] Teufel et al. MEGAN: Multi-Explanation Graph Attention Network. XAI'22

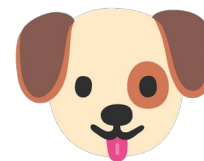
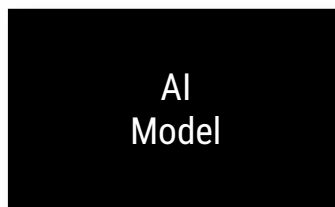
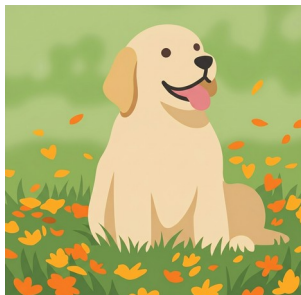


Paper Online

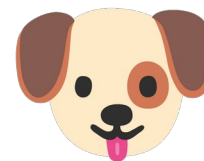
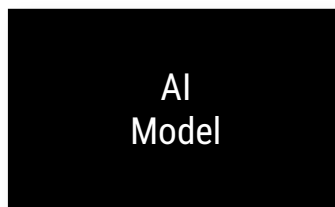
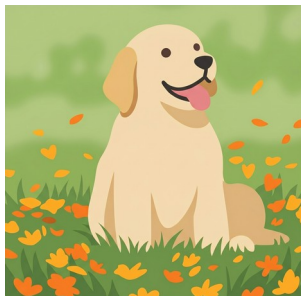


MEGAN directly predicts the main target value in addition to multiple attributional explanations

What are Counterfactual explanations?

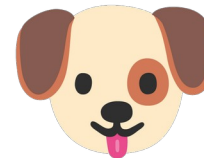
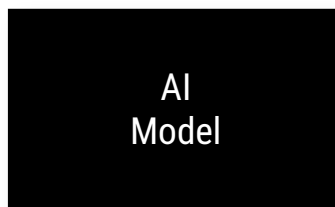
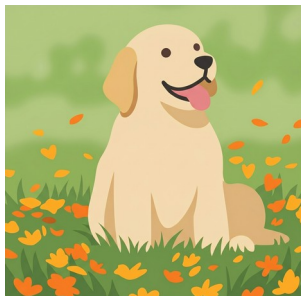


What are Counterfactual explanations?

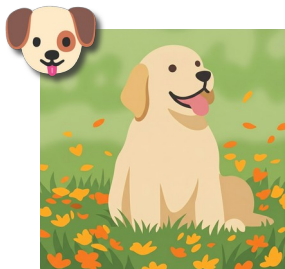


What is the *minimal change* to the *input*, which causes the *maximal deviation* of the *prediction outcome*?

What are Counterfactual explanations?

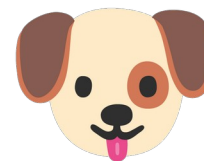
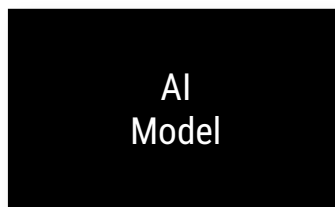
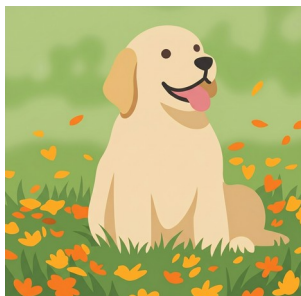


What is the *minimal change* to the *input*, which causes the *maximal deviation* of the *prediction outcome*?

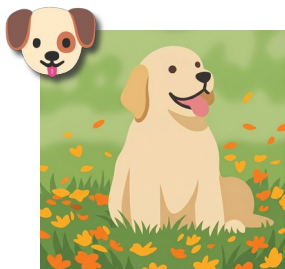


Original Image

What are Counterfactual explanations?



What is the *minimal change* to the *input*, which causes the *maximal deviation* of the *prediction outcome*?

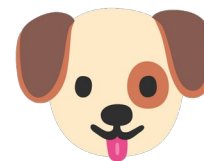
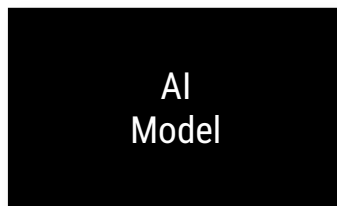
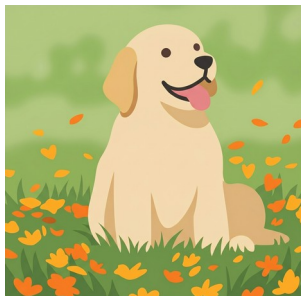


Original Image

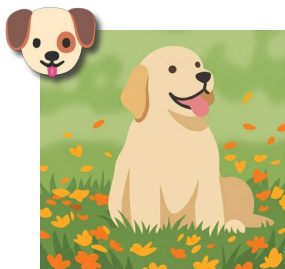


with cat ears

What are Counterfactual explanations?



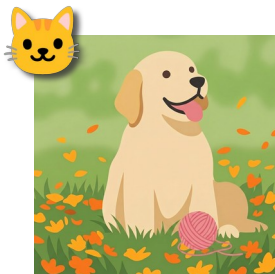
What is the *minimal change* to the *input*, which causes the *maximal deviation* of the *prediction outcome*?



Original Image

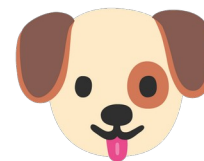
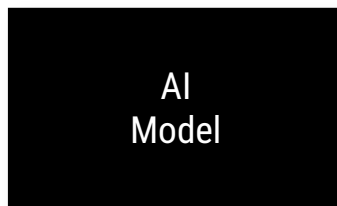
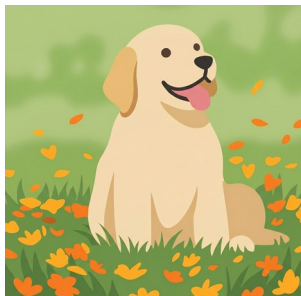


with cat ears

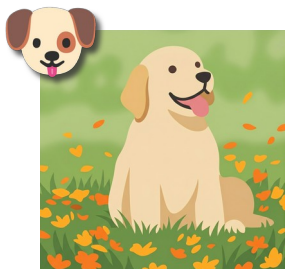


with yarn ball

What are Counterfactual explanations?



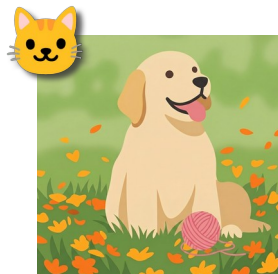
What is the *minimal change* to the *input*, which causes the *maximal deviation* of the *prediction outcome*?



Original Image



with cat ears



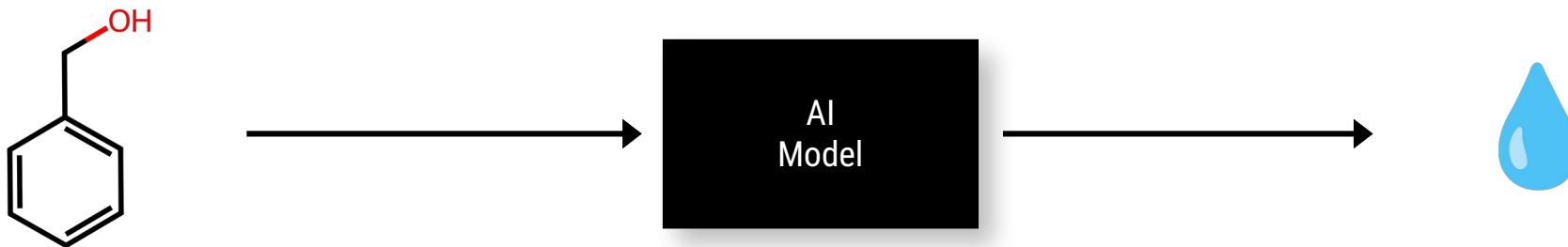
with yarn ball



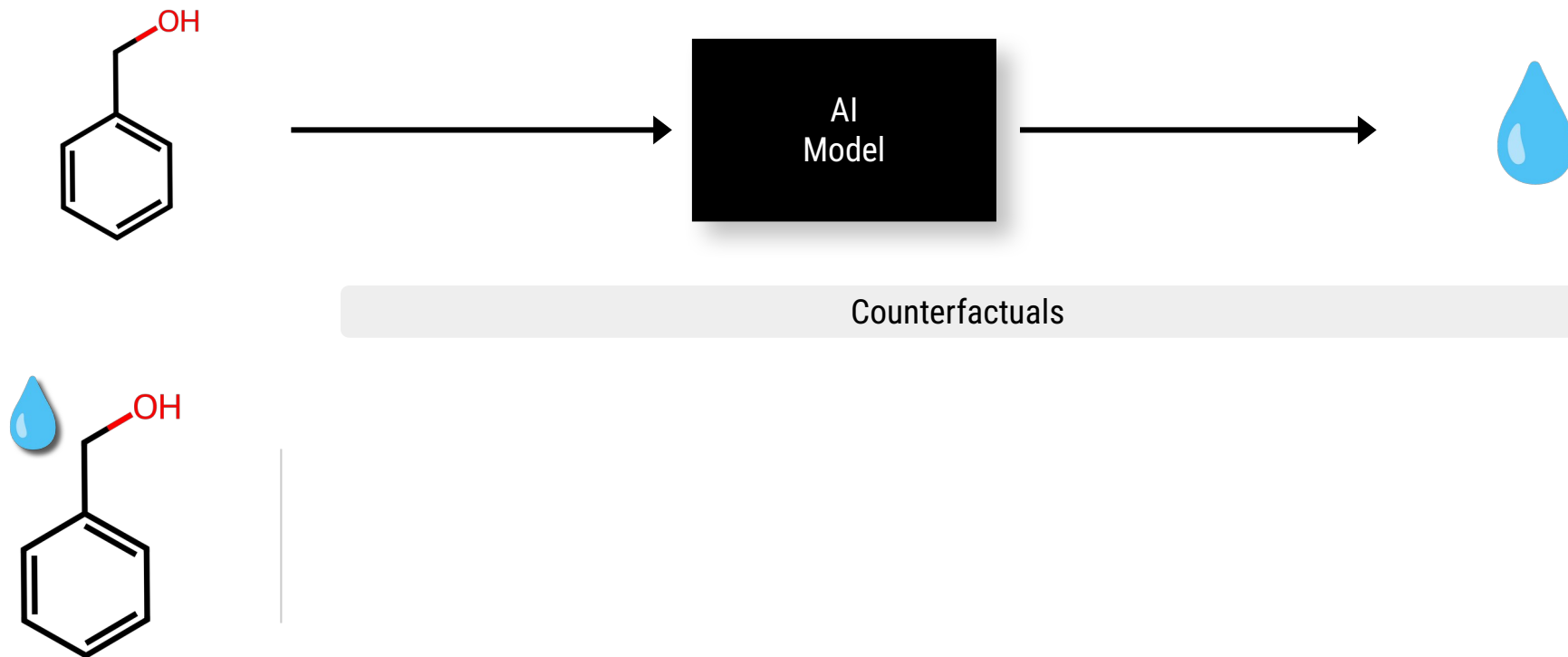
with fish



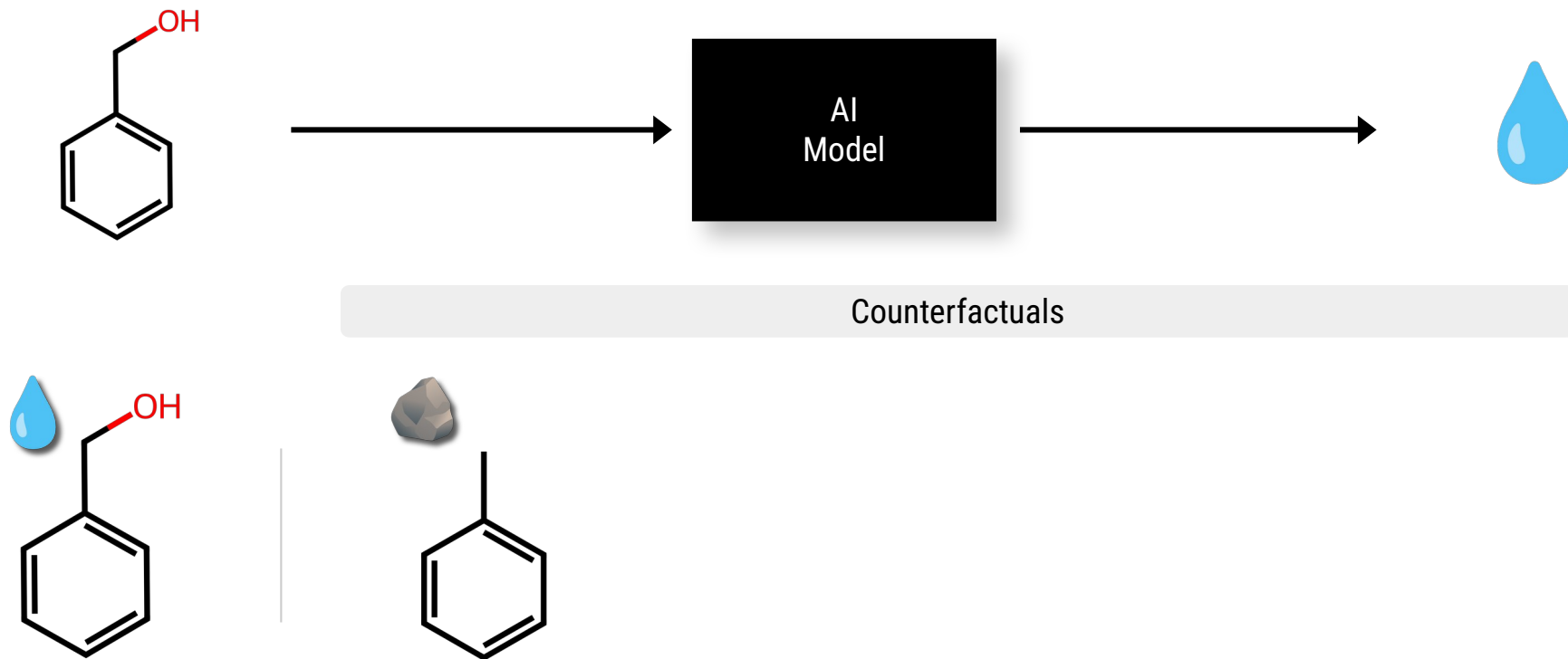
Counterfactual explanations for molecules



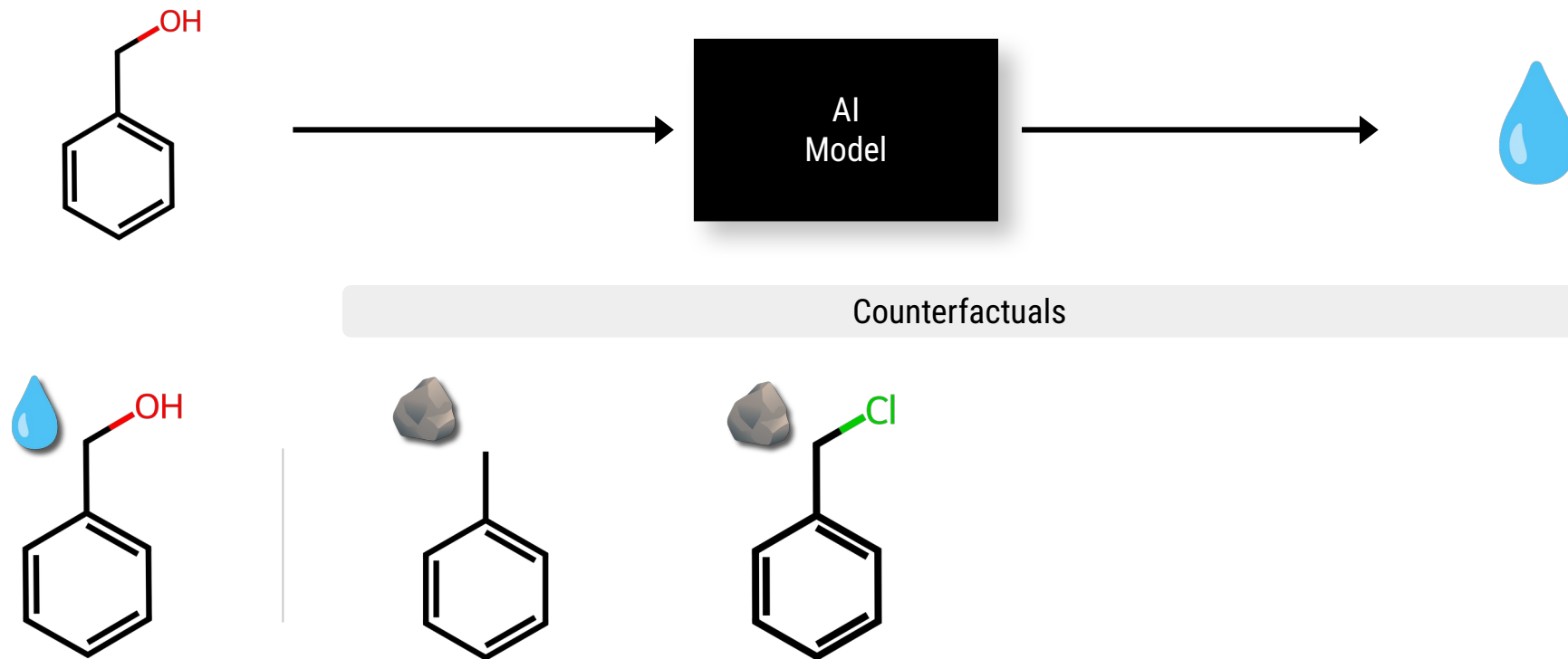
Counterfactual explanations for molecules



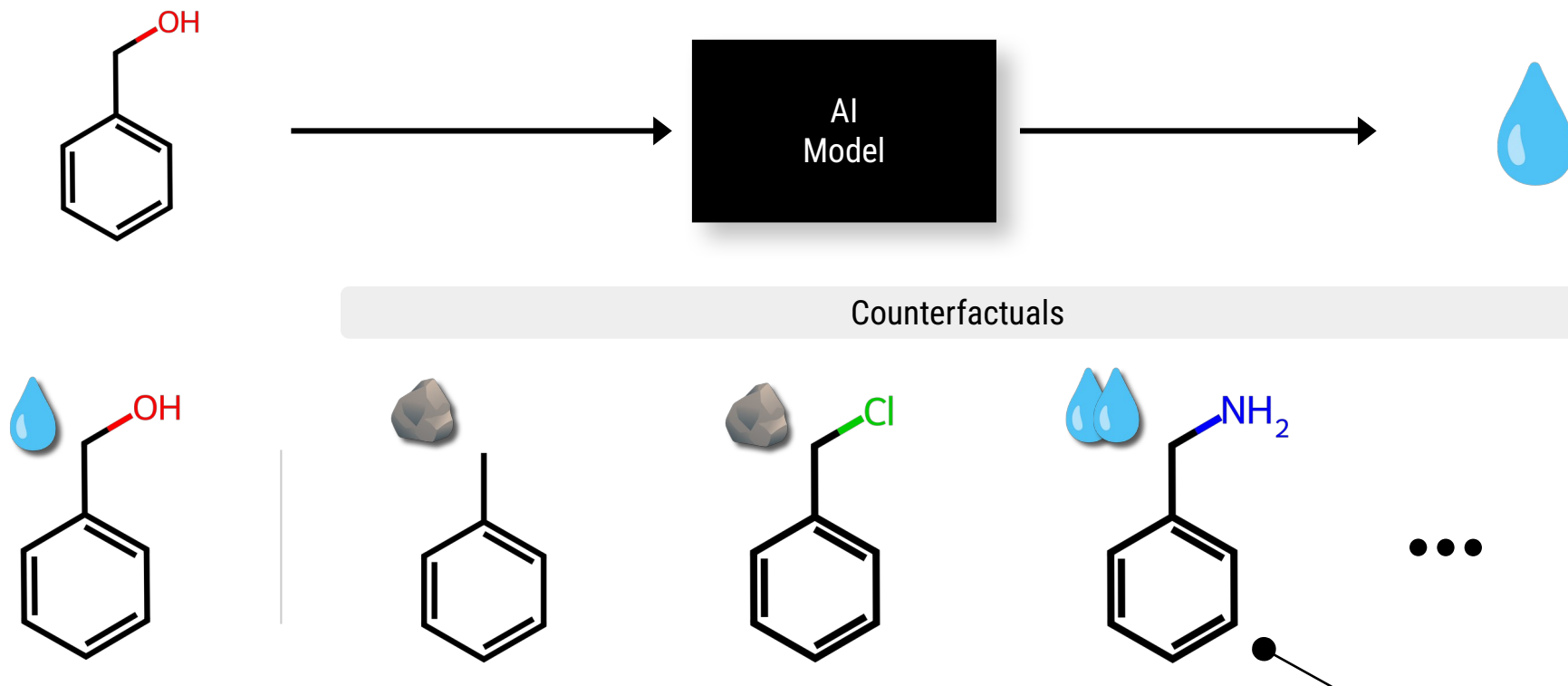
Counterfactual explanations for molecules



Counterfactual explanations for molecules

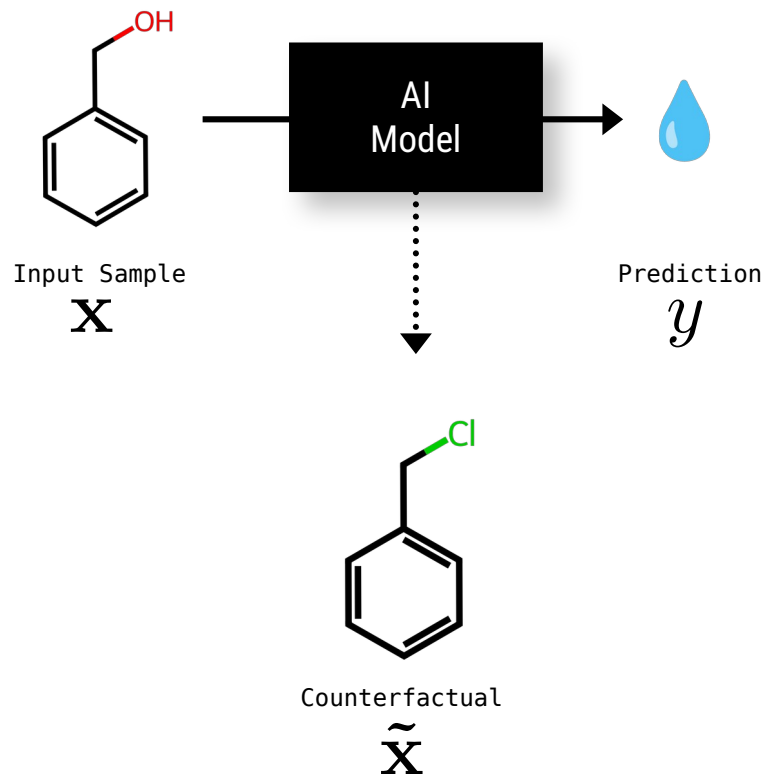


Counterfactual explanations for molecules

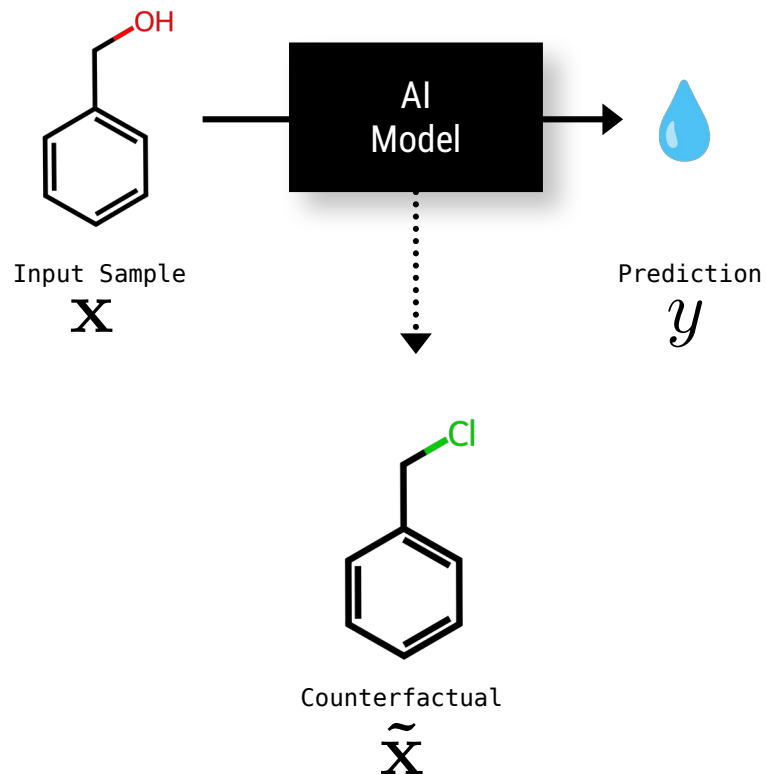


For regression tasks, "maximum output deviation" could be *increasing* or *decreasing*!

The counterfactual optimization problem

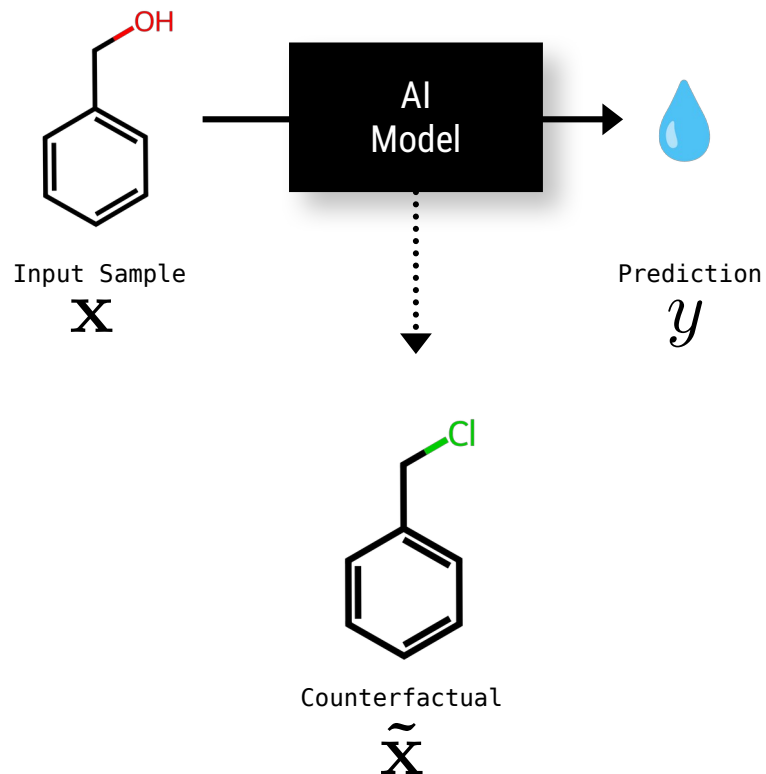


The counterfactual optimization problem



$$\tilde{\mathbf{x}} = \arg \min_{\mathbf{x}'} (d(\mathbf{x}, \mathbf{x}') - \lambda \cdot |y - y'|)$$

The counterfactual optimization problem

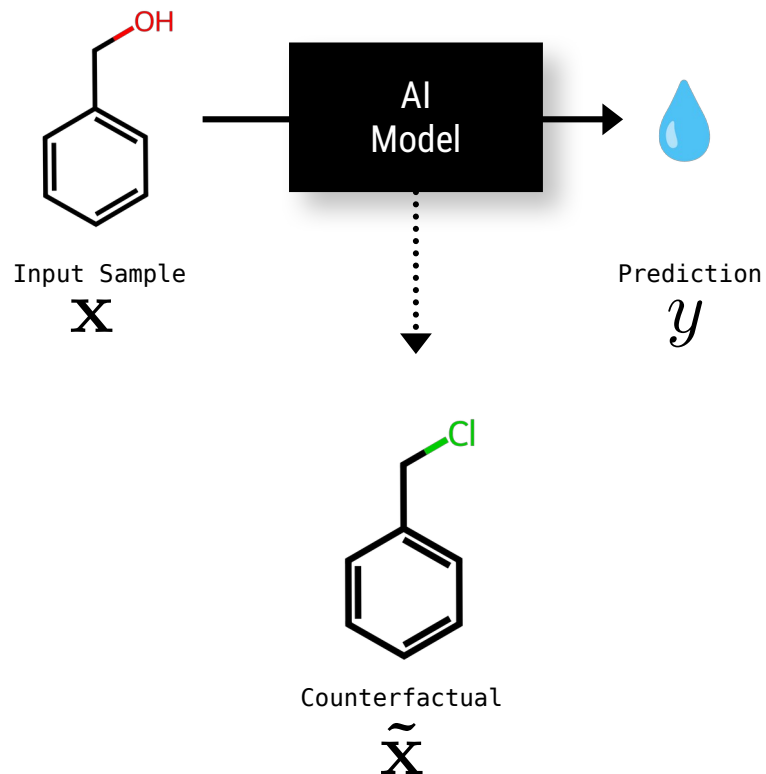


Minimize structural distance to original sample



$$\tilde{\mathbf{x}} = \arg \min_{\mathbf{x}'} (d(\mathbf{x}, \mathbf{x}') - \lambda \cdot |y - y'|)$$

The counterfactual optimization problem



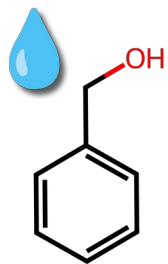
Minimize structural distance to original sample



$$\tilde{\mathbf{x}} = \arg \min_{\mathbf{x}'} (d(\mathbf{x}, \mathbf{x}') - \lambda \cdot |y - y'|)$$

Maximize the prediction deviation

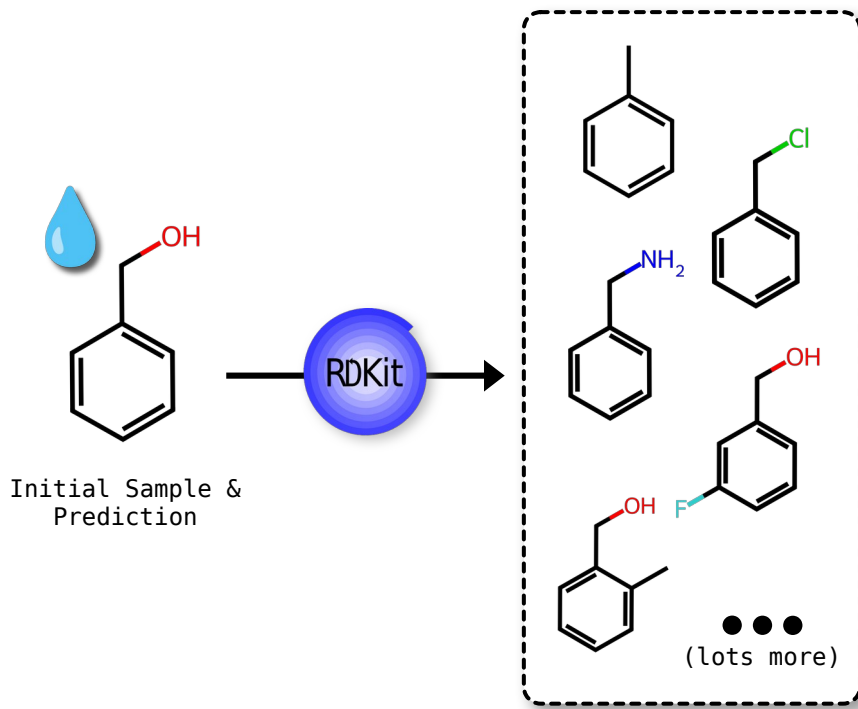
Generating molecular counterfactuals by total enumeration



Initial Sample &
Prediction

Generating molecular counterfactuals by total enumeration

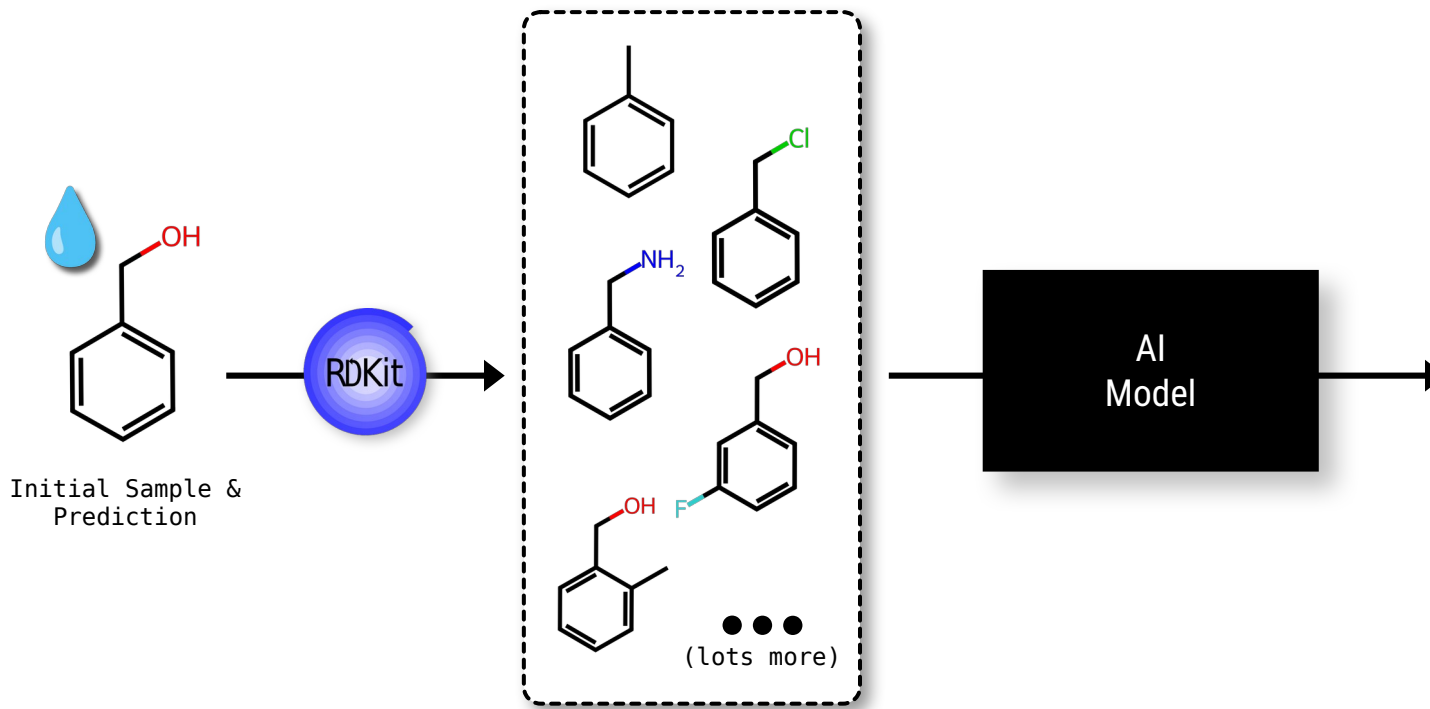
I Obtain all chemically feasible single-edit modifications



Generating molecular counterfactuals by total enumeration

I Obtain all chemically feasible single-edit modifications

II Query the model for predictions

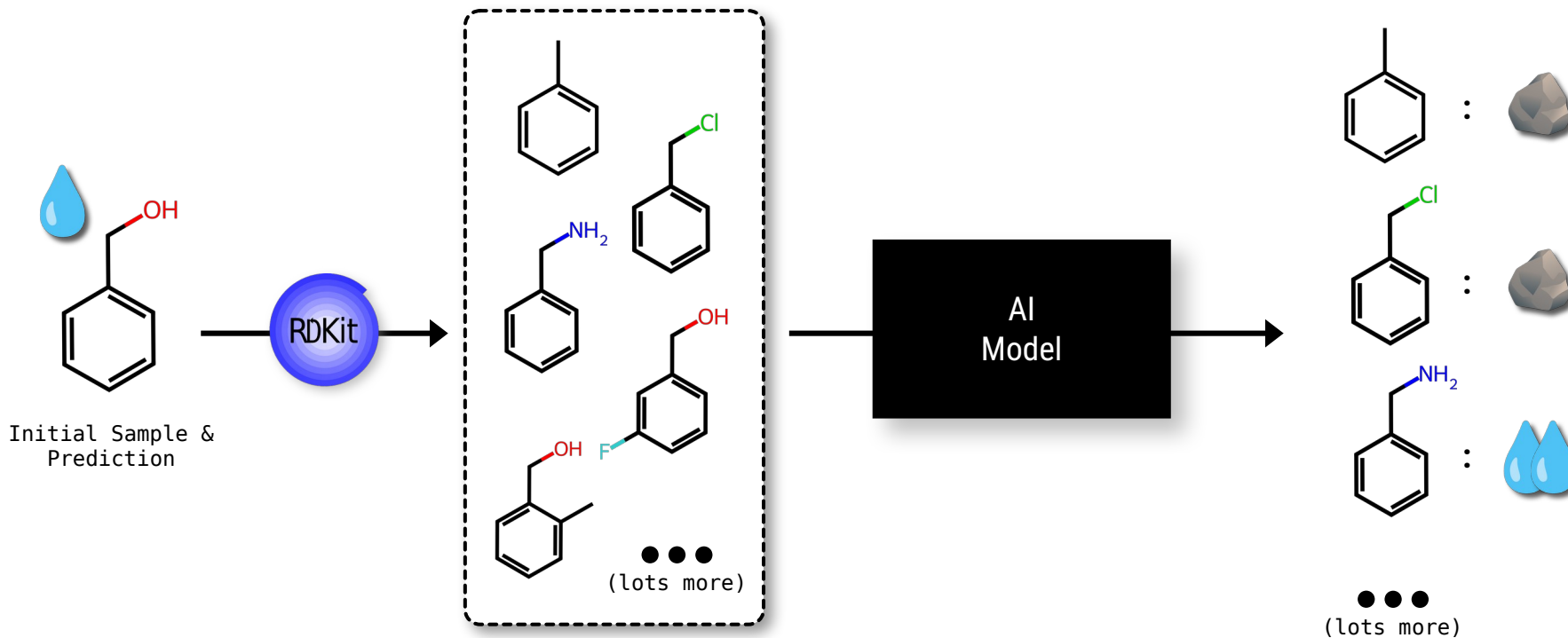


Generating molecular counterfactuals by total enumeration

I Obtain all chemically feasible single-edit modifications

II Query the model for predictions

III Rank by deviation and select top-k as counterfactuals

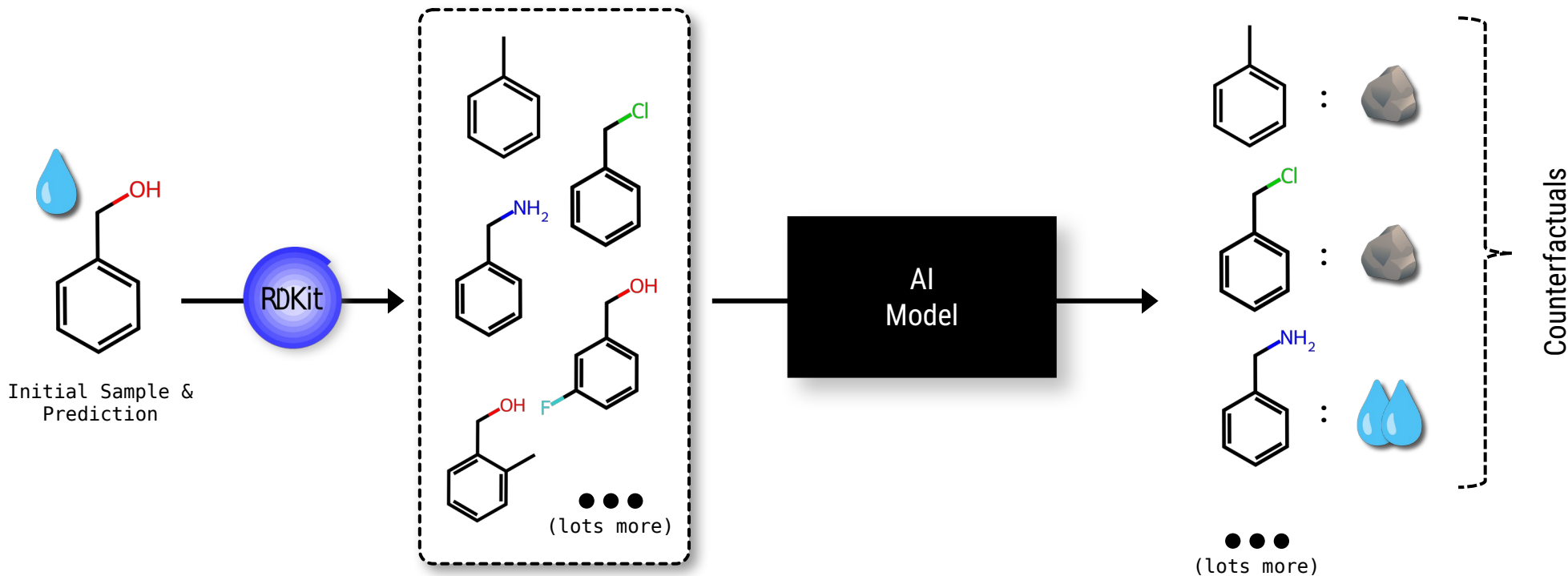


Generating molecular counterfactuals by total enumeration

I Obtain all chemically feasible single-edit modifications

II Query the model for predictions

III Rank by deviation and select top-k as counterfactuals



GUIDE

Generating molecular counterfactuals by total enumeration

```
from vgd_counterfactuals.generate.molecules import get_neighborhood

# 0 - Obtaining original prediction

# 1 - Generating all possible neighbors

# 2 - Obtaining model predictions

# 3 - ordering and choosing top-5 counterfactuals
```

GUIDE

Generating molecular counterfactuals by total enumeration

```
from vgd_counterfactuals.generate.molecules import get_neighborhood
```

```
# 0 - Obtaining original prediction
```

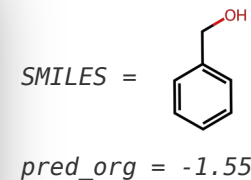
```
SMILES: str = "C1=CC=CC=C1CO" # benzylalcohol
```

```
pred_org: float = model.predict_smiles(SMILES)
```

```
# 1 - Generating all possible neighbors
```

```
# 2 - Obtaining model predictions
```

```
# 3 - ordering and choosing top-5 counterfactuals
```



GUIDE

Generating molecular counterfactuals by total enumeration

```
from vgd_counterfactuals.generate.molecules import get_neighborhood
```

```
# 0 - Obtaining original prediction
```

```
SMILES: str = "C1=CC=CC=C1CO" # benzylalcohol
```

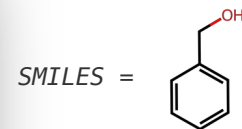
```
pred_org: float = model.predict_smiles(SMILES)
```

```
# 1 - Generating all possible neighbors
```

```
neighbors: list[str] = get_neighborhood(SMILES)
```

```
# 2 - Obtaining model predictions
```

```
# 3 - ordering and choosing top-5 counterfactuals
```



pred_org = -1.55

neighbors = ["C1=CC=CC=C1CN", "C1=CN=CC=C1CO",
"C1=CC(C)=CC=C1CO", ... +41]

GUIDE

Generating molecular counterfactuals by total enumeration

```
from vgd_counterfactuals.generate.molecules import get_neighborhood
```

```
# 0 - Obtaining original prediction
```

```
SMILES: str = "C1=CC=CC=C1CO" # benzylalcohol
```

```
pred_org: float = model.predict_smiles(SMILES)
```

```
# 1 - Generating all possible neighbors
```

```
neighbors: list[str] = get_neighborhood(SMILES)
```

```
# 2 - Obtaining model predictions
```

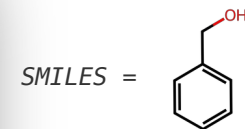
```
predictions: list[float] = [
```

```
    model.predict_smiles(smi)
```

```
    for smi in neighbors
```

```
]
```

```
# 3 - ordering and choosing top-5 counterfactuals
```



pred_org = -1.55

neighbors = ["C1=CC=CC=C1CN", "C1=CN=CC=C1CO",
"C1=CC(C)=CC=C1CO", ... +41]

Predictions = [-0.53, -0.57,
-2.11, ... +41]

GUIDE

Generating molecular counterfactuals by total enumeration

```
from vgd_counterfactuals.generate.molecules import get_neighborhood
```

```
# 0 - Obtaining original prediction
```

```
SMILES: str = "C1=CC=CC=C1CO" # benzylalcohol
pred_org: float = model.predict_smiles(SMILES)
```

```
# 1 - Generating all possible neighbors
```

```
neighbors: list[str] = get_neighborhood(SMILES)
```

```
# 2 - Obtaining model predictions
```

```
predictions: list[float] = [
    model.predict_smiles(smi)
    for smi in neighbors
]
```

```
# 3 - ordering and choosing top-5 counterfactuals
```

```
counterfactuals: list[tuple[str, float]] = list(sorted(
    zip(neighbors, predictions)
),
    key=lambda t: -abs(t[1] - pred_org)
)[:5]
```



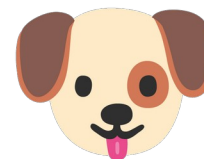
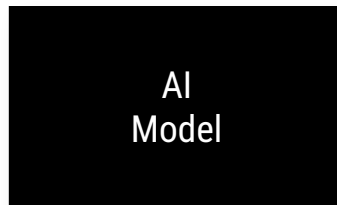
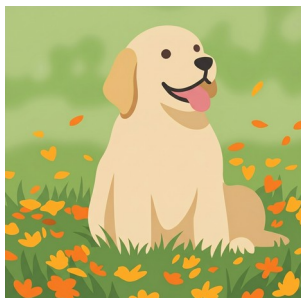
pred_org = -1.55

neighbors = ["C1=CC=CC=C1CN", "C1=CN=CC=C1CO",
"C1=CC(C)=CC=C1CO", ... +41]

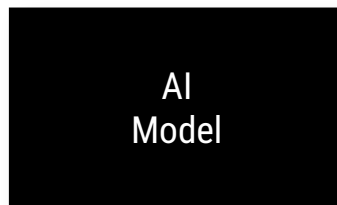
Predictions = [-0.53, -0.57,
-2.11, ... +41]

counterfactuals = [
("C1=CC=CC(C)=C1CO", -3.22),
("C1=CC=CC(F)=C1CO", -2.74),
("C1=CC=CC(S)=C1CO", -2.69),
("C1=CC=CC(=S)1CO", -2.61),
("C1=CC=CC(C)=C1CO", -2.59)
]

What is Uncertainty Quantification? And why is it useful?



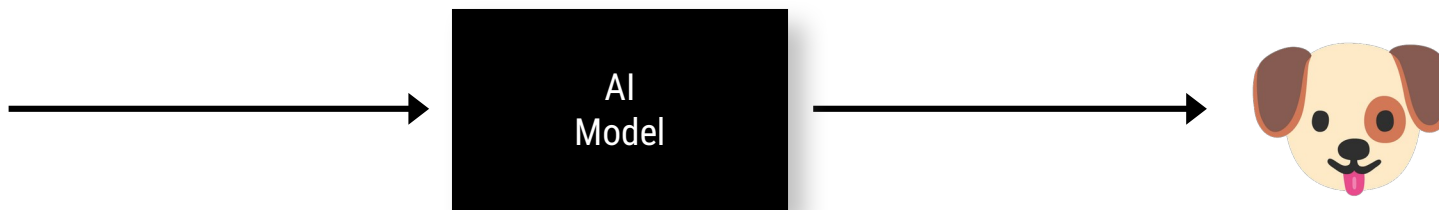
What is Uncertainty Quantification? And why is it useful?



What is Uncertainty Quantification? And why is it useful?



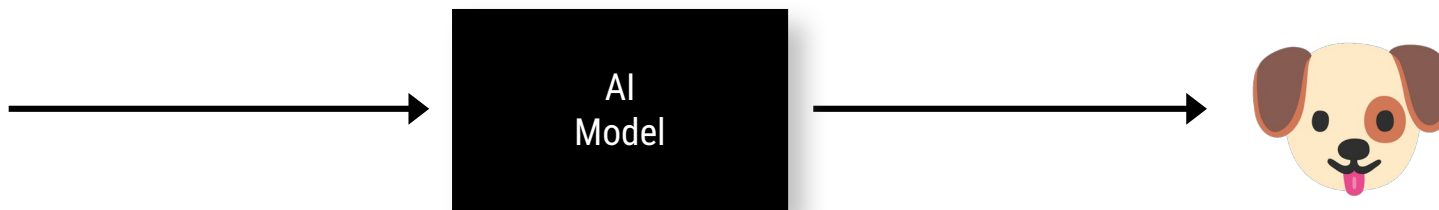
Out-of-distribution
Sample



What is Uncertainty Quantification? And why is it useful?



Out-of-distribution
Sample

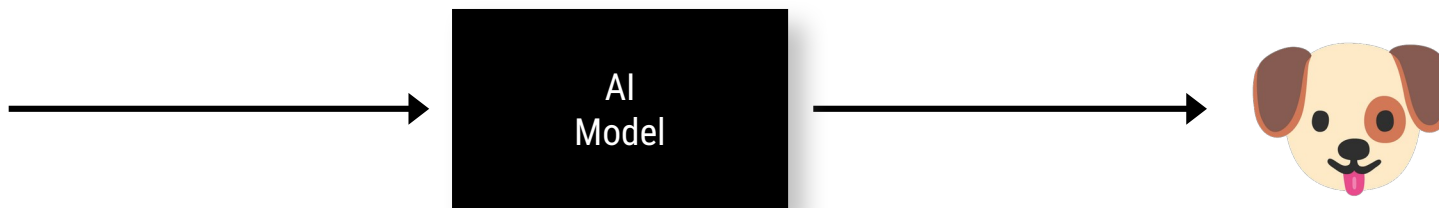


Models will always make a prediction -- even for *out-of-distribution samples* -- potentially leading to harmful outcomes

What is Uncertainty Quantification? And why is it useful?



Out-of-distribution
Sample



Models will always make a prediction -- even for *out-of-distribution samples* -- potentially leading to harmful outcomes

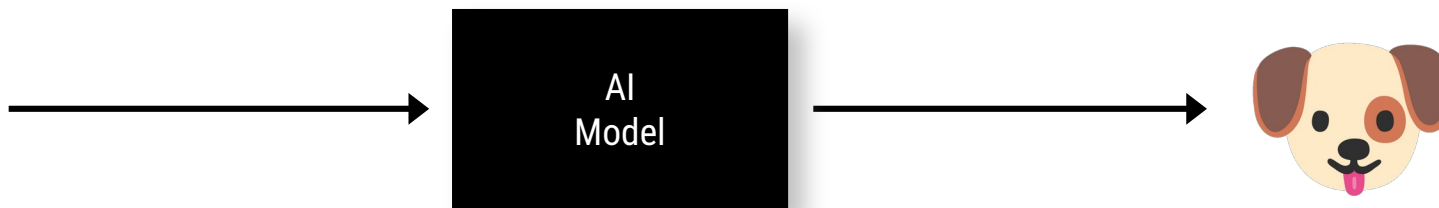


Training distribution: Only cats and dogs

What is Uncertainty Quantification? And why is it useful?



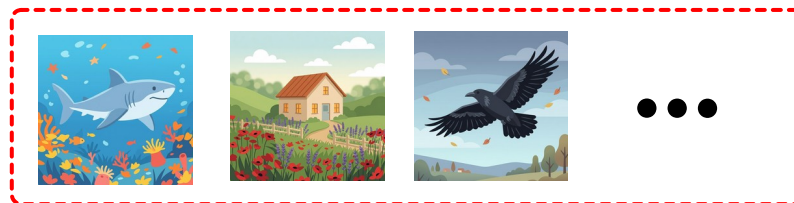
Out-of-distribution
Sample



Models will always make a prediction -- even for *out-of-distribution samples* -- potentially leading to harmful outcomes



Training distribution: Only cats and dogs

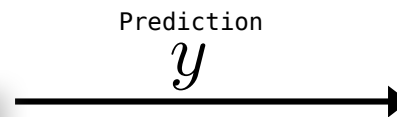
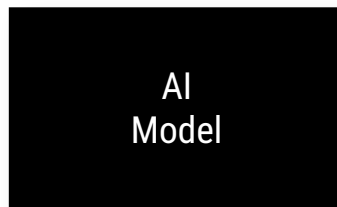
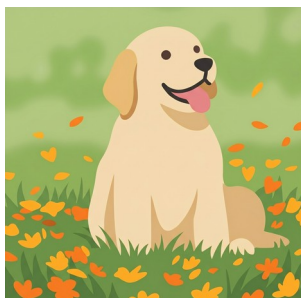


Everything else: Out of distribution

What is Uncertainty Quantification? And why is it useful? II



Adding an additional quantification of "*model uncertainty*" as an auxiliary output of the prediction process



Prediction

y



Uncertainty
Quantification

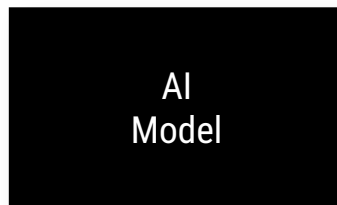
σ



What is Uncertainty Quantification? And why is it useful? II



Adding an additional quantification of “*model uncertainty*” as an auxiliary output of the prediction process

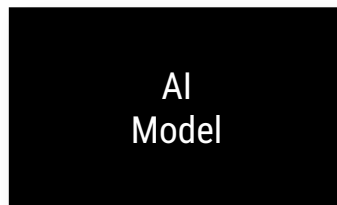


Highly uncertain!

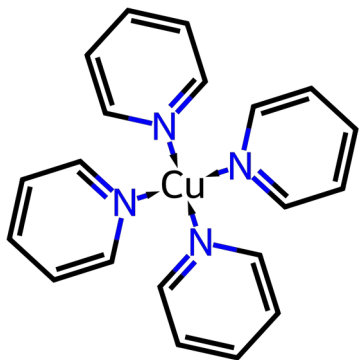
What is Uncertainty Quantification? And why is it useful? II



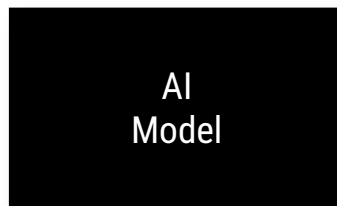
Adding an additional quantification of "*model uncertainty*" as an auxiliary output of the prediction process



Highly uncertain!



OOD Molecule (MOF)

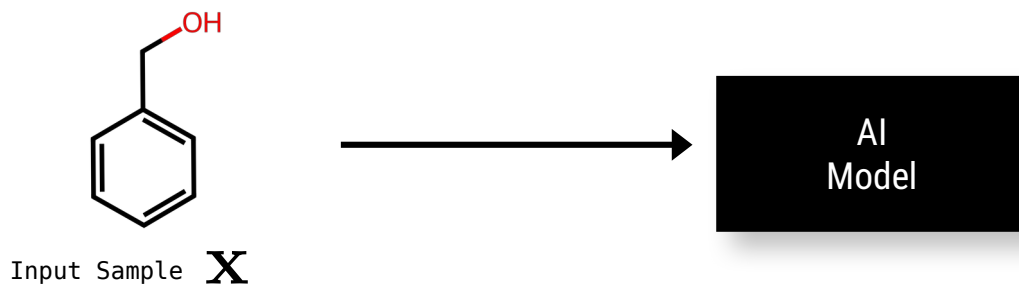


Highly uncertain!

Mean Variance Estimation (MVE) models

[3] Nix and Weigend. *Estimating the Mean and Variance of a Target Probability Distribution*. ICNN'94 [🔗](#)

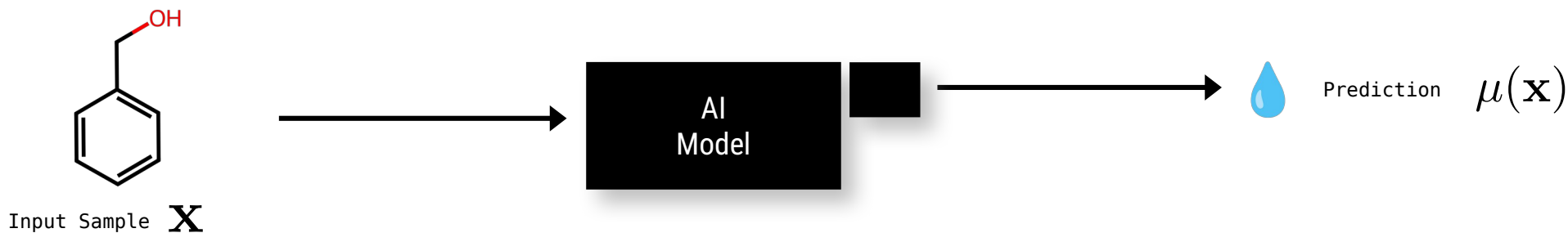
Mean Variance Estimation (MVE)



Mean Variance Estimation (MVE) models

[3] Nix and Weigend. *Estimating the Mean and Variance of a Target Probability Distribution*. ICNN'94 [🔗](#)

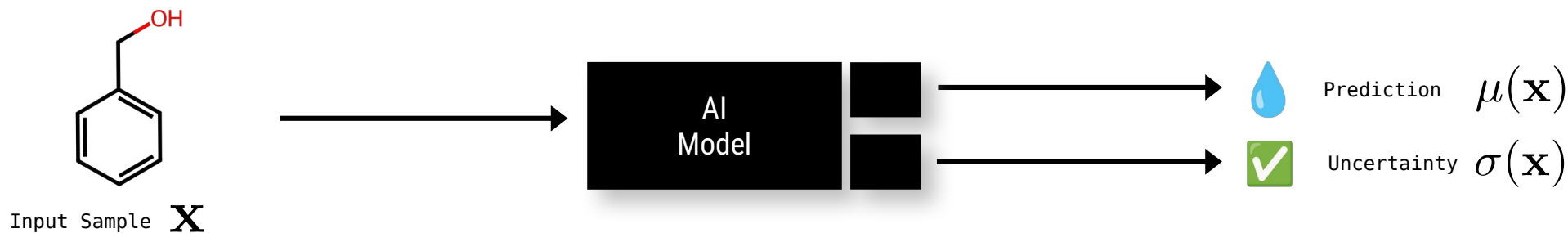
Mean Variance Estimation (MVE)



Mean Variance Estimation (MVE) models

[3] Nix and Weigend. *Estimating the Mean and Variance of a Target Probability Distribution*. ICNN'94 [link](#)

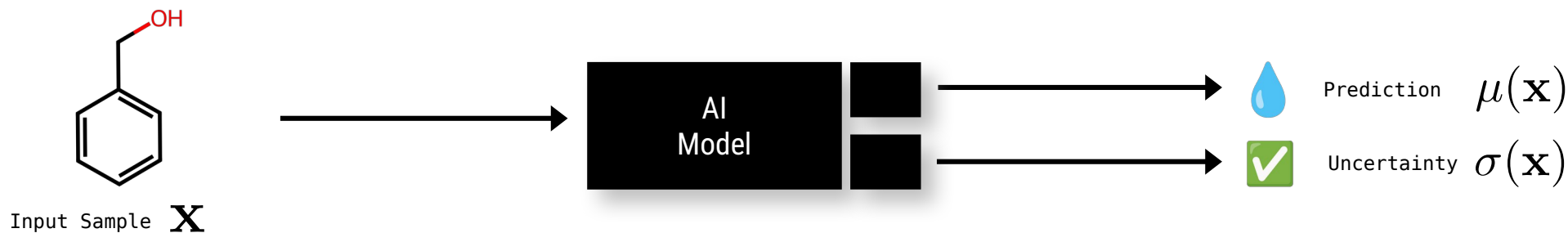
Mean Variance Estimation (MVE)



Mean Variance Estimation (MVE) models

[3] Nix and Weigend. *Estimating the Mean and Variance of a Target Probability Distribution*. ICNN'94 [🔗](#)

Mean Variance Estimation (MVE)

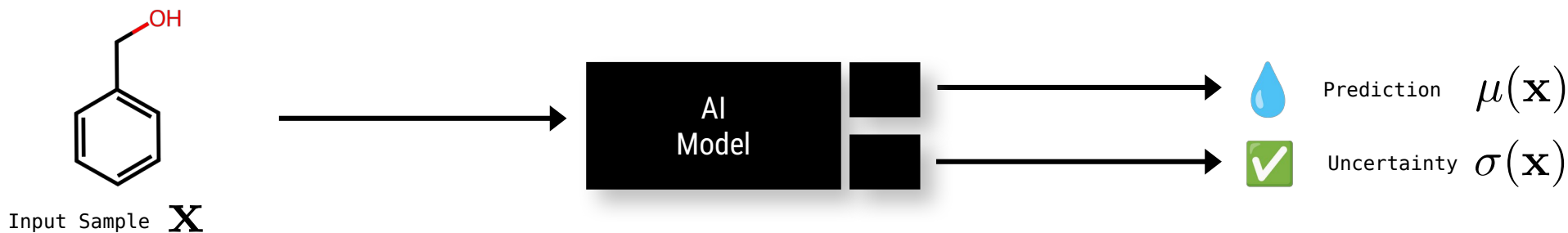


Training the network to predict the mean and the variance at the same time

Mean Variance Estimation (MVE) models

[3] Nix and Weigend. *Estimating the Mean and Variance of a Target Probability Distribution*. ICNN'94 [🔗](#)

Mean Variance Estimation (MVE)



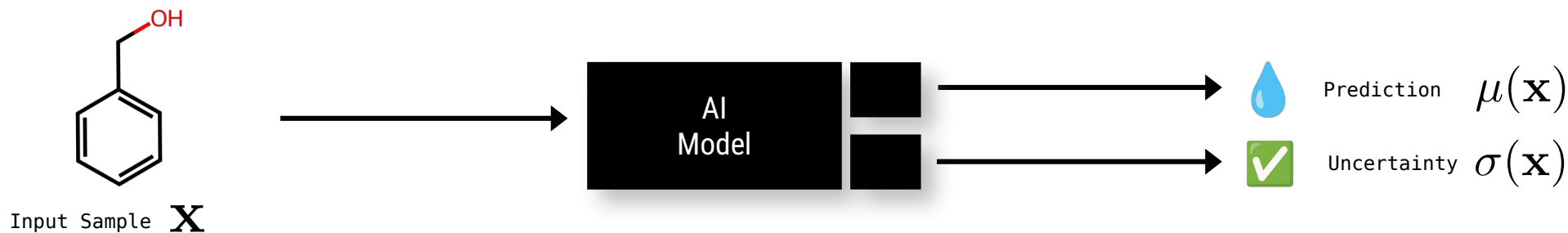
Training the network to predict the mean and the variance at the same time

$$\mathcal{L} = \frac{(y - \mu(\mathbf{x}))^2}{2\sigma(\mathbf{x})^2} + \frac{\log \sigma(\mathbf{x})^2}{2}$$

Mean Variance Estimation (MVE) models

[3] Nix and Weigend. *Estimating the Mean and Variance of a Target Probability Distribution*. ICNN'94 [🔗](#)

Mean Variance Estimation (MVE)



Training the network to predict the mean and the variance at the same time

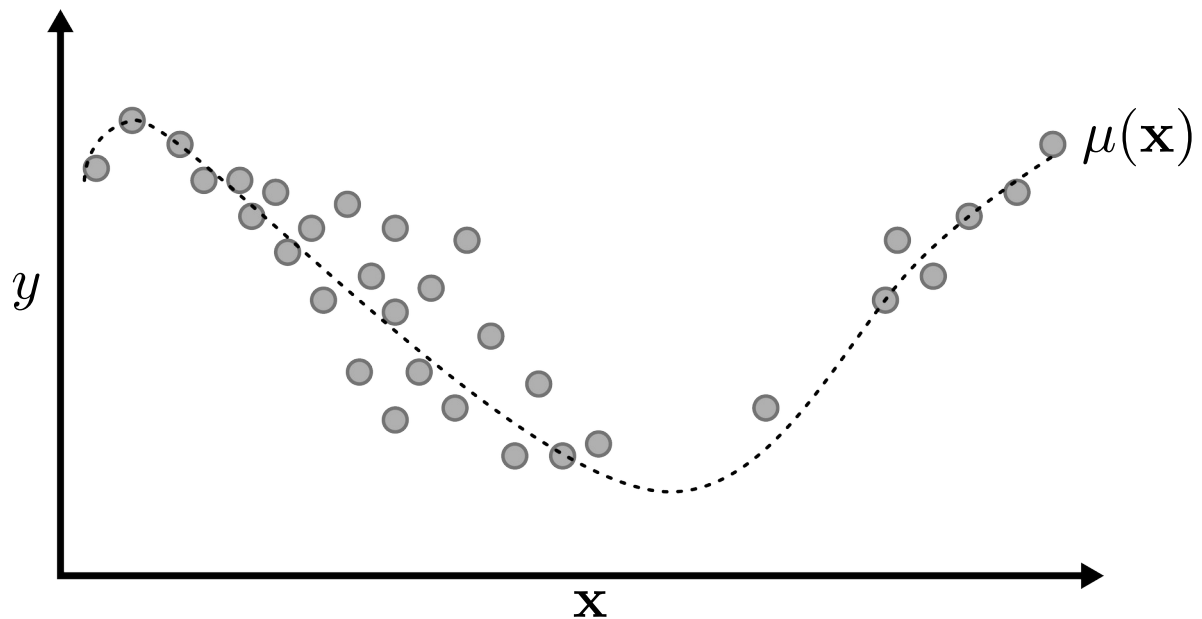
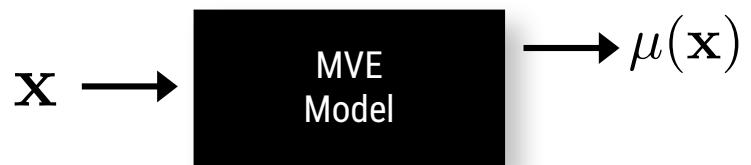
2 methods to minimize the loss:

Assign the correct prediction

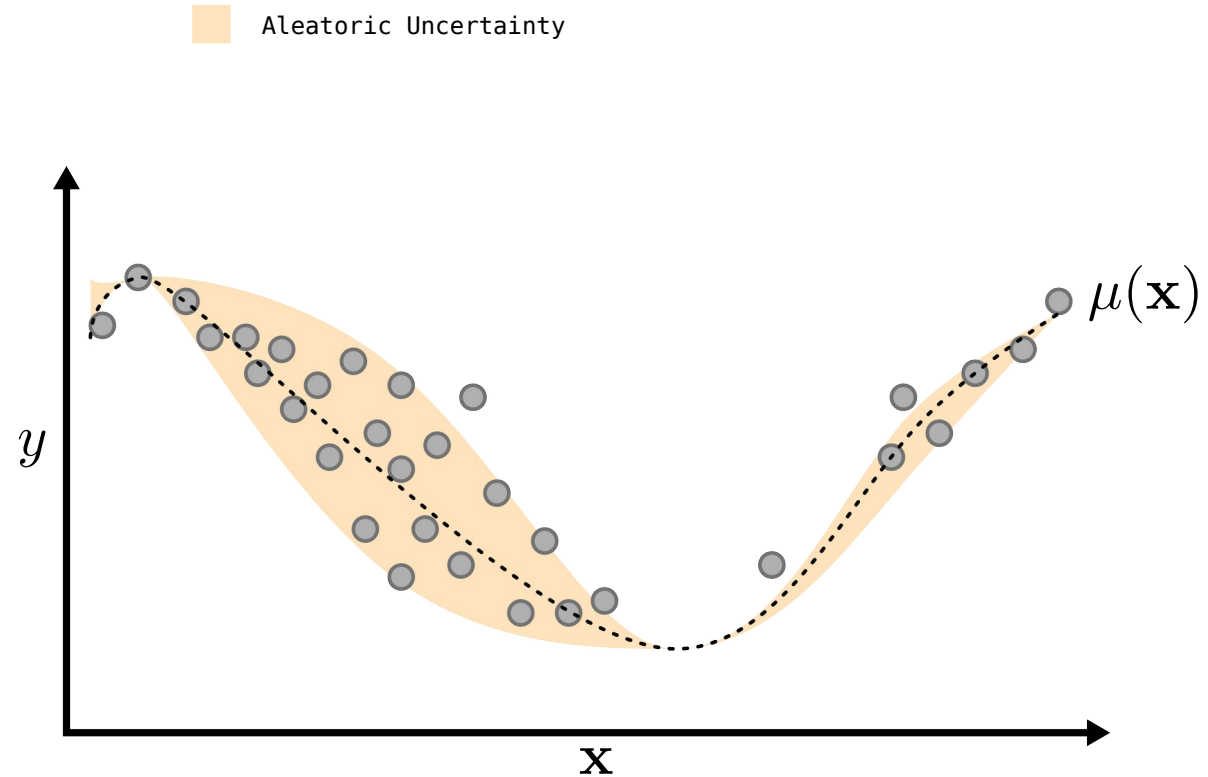
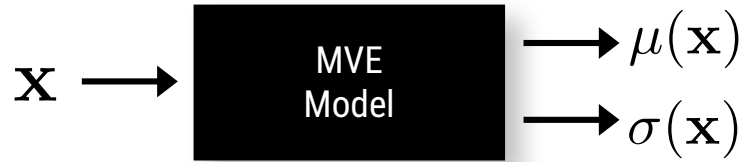
$$\mathcal{L} = \frac{(y - \mu(\mathbf{x}))^2}{2\sigma(\mathbf{x})^2} + \frac{\log \sigma(\mathbf{x})^2}{2}$$

Assign a high variance

Mean Variance Estimation (MVE) models II

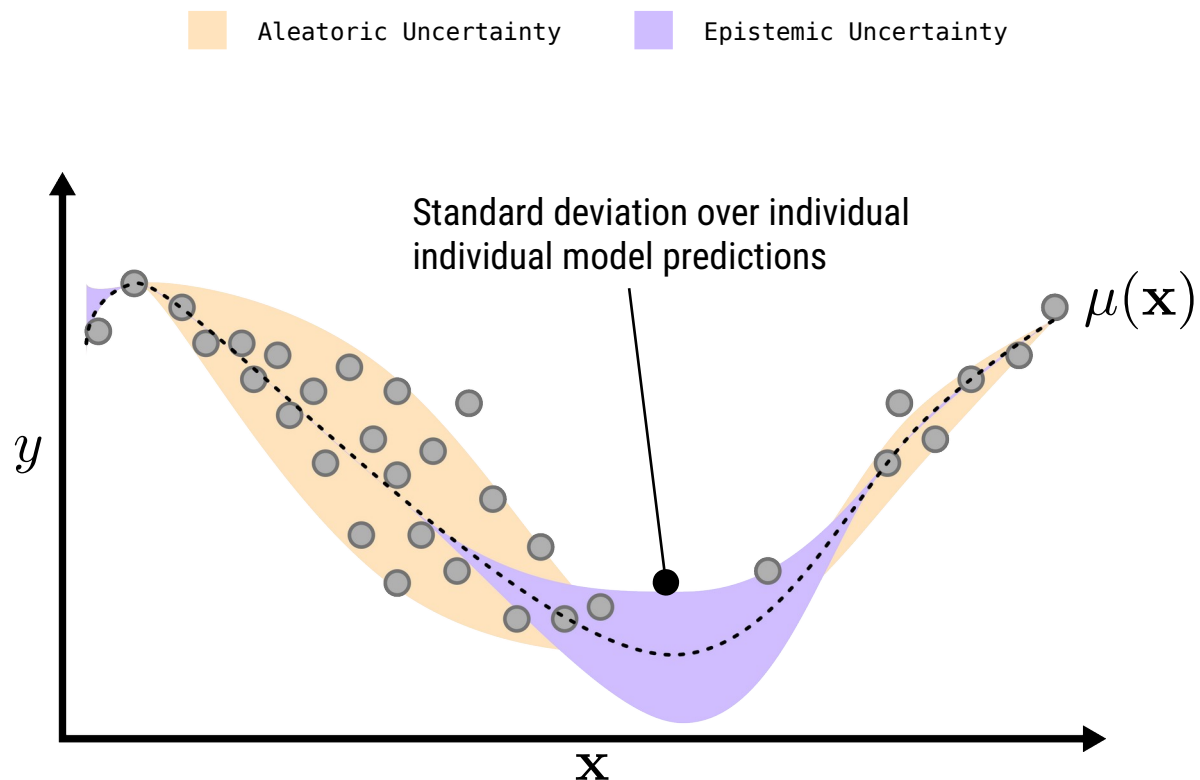
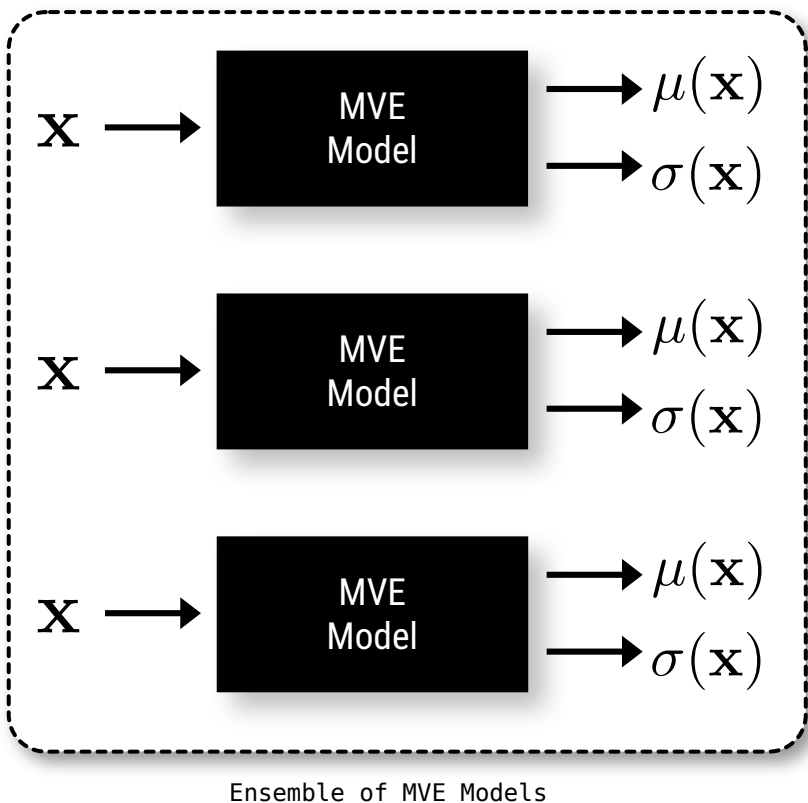


Mean Variance Estimation (MVE) models II



Uncertainty quantification using deep ensembles

[4] Lakshminarayanan et al. *Simple and Scalable Predictive Uncertainty using Deep Ensembles*. NIPS'17 [link](#)



GUIDE

✦ Uncertainty quantification using deep ensembles

```
import torch
import lightning as L

class MveModel(L.LigthingModule):

    # ...

    def train_step(self, data: Batch, batch_idx: int) → torch.Tensor:

        # 1 - obtain the prediction
        # The "forward" call of the modified network should return the mean and
        # the standard deviation as two separate outputs.

        # 2 - calculate NLL loss

    return loss
```

GUIDE

✦ Uncertainty quantification using deep ensembles

```
import torch
import lightning as L

class MveModel(L.LightningModule):

    # ...

    def train_step(self, data: Batch, batch_idx: int) → torch.Tensor:

        # 1 - obtain the prediction
        # The "forward" call of the modified network should return the mean and
        # the standard deviation as two separate outputs.
        mu, sig = self.forward(x)

        # 2 - calculate NLL loss

    return loss
```

def forward():



GUIDE

✦ Uncertainty quantification using deep ensembles

```
import torch
import lightning as L

class MveModel(L.LightningModule):

    # ...

    def train_step(self, data: Batch, batch_idx: int) → torch.Tensor:

        # 1 - obtain the prediction
        # The "forward" call of the modified network should return the mean and
        # the standard deviation as two separate outputs.
        mu, sig = self.forward(x)

        # 2 - calculate NLL loss
        loss = (0.5 * (data.y - mu) / sig) ** 2 + torch.log(sig).mean()

        return loss
```

def forward():



$$\mathcal{L} = \frac{(y - \mu(\mathbf{x}))^2}{2\sigma(\mathbf{x})^2} + \frac{\log \sigma(\mathbf{x})^2}{2}$$

GUIDE

✦ Uncertainty quantification using deep ensembles II

```
import numpy as np
```

```
SMILES = "C1=CC=CC=C1CO"
```

```
mve_models: list[MveModel] = # ...
```

```
# 1 - Obtain mean and standard deviations for all the models
```

```
# 2 - Caculate the prediction as the average of the individual  
#     mean predictions
```

```
# 3 - Calculate the overall uncertainty
```

A list of multiple trained models

GUIDE

✦ Uncertainty quantification using deep ensembles II

```
import numpy as np

SMILES = "C1=CC=CC=C1CO"
mve_models: list[MveModel] = # ...

# 1 - Obtain mean and standard deviations for all the models
mus, sigs = zip(*[
    model.predict_smiles(SMILES) # returns (mu, sig) tuple
    for model in mve_models
])

# 2 - Caculate the prediction as the average of the individual
#     mean predictions

# 3 - Calculate the overall uncertainty
```

A list of multiple trained models

GUIDE

✦ Uncertainty quantification using deep ensembles II

```
import numpy as np

SMILES = "C1=CC=CC=C1CO"
mve_models: list[MveModel] = # ...

# 1 - Obtain mean and standard deviations for all the models
mus, sigs = zip(*[
    model.predict_smiles(SMILES) # returns (mu, sig) tuple
    for model in mve_models
])

# 2 - Caculate the prediction as the average of the individual
#     mean predictions
pred: float = np.mean(mus)

# 3 - Calculate the overall uncertainty
```

A list of multiple trained models

$$\mu_* = \frac{1}{M} \sum_m \mu_m(\mathbf{x})$$

GUIDE

✦ Uncertainty quantification using deep ensembles II

```
import numpy as np

SMILES = "C1=CC=CC=C1CO"
mve_models: list[MveModel] = # ...

# 1 - Obtain mean and standard deviations for all the models
mus, sigs = zip(*[
    model.predict_smiles(SMILES) # returns (mu, sig) tuple
    for model in mve_models
])

# 2 - Caculate the prediction as the average of the individual
#     mean predictions
pred: float = np.mean(mus)

# 3 - Calculate the overall uncertainty
unc: float = np.sqrt( (1 / len(sigs)) * sum([
    (sig ** 2 + mu ** 2) - pred ** 2
    for mu, sig in zip(mus, sigs)
]) )
```

A list of multiple trained models

$$\mu_* = \frac{1}{M} \sum_m \mu_m(\mathbf{x})$$

$$\sigma_*^2 = \frac{1}{M} \sum_m (\mu_m(\mathbf{x})^2 + \sigma_m(\mathbf{x})^2 - \mu_*^2)$$

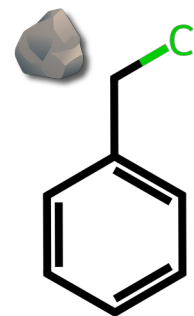
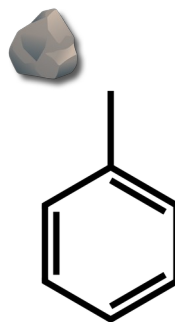
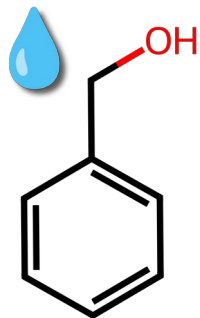
Combining uncertainty quantification & other explanations!

[5] Teufel et al. *Improving counterfactual truthfulness [...] through uncertainty quantification*. XAI'25 

Combining uncertainty quantification & other explanations!

[5] Teufel et al. *Improving counterfactual truthfulness [...] through uncertainty quantification*. XAI'25 [🔗](#)

Coming back to counterfactuals

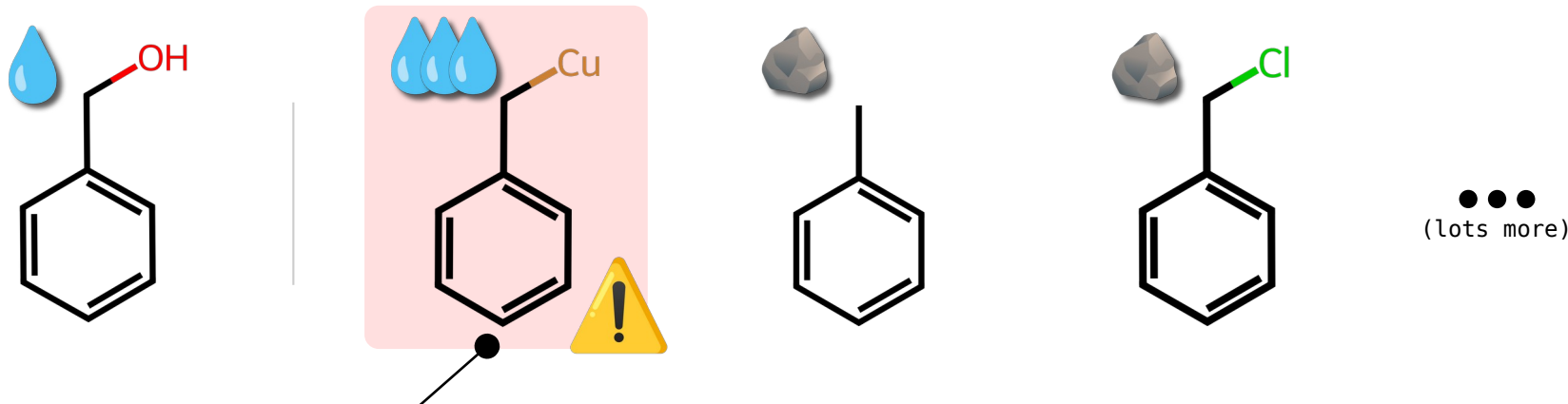


● ● ●
(lots more)

Combining uncertainty quantification & other explanations!

[5] Teufel et al. *Improving counterfactual truthfulness [...] through uncertainty quantification*. XAI'25 [🔗](#)

Coming back to counterfactuals

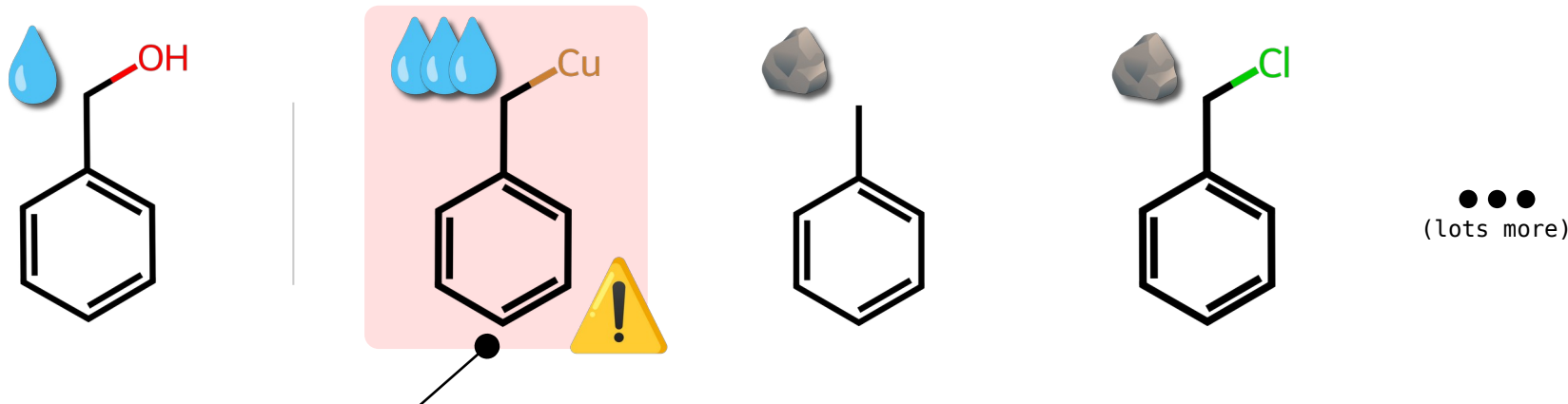


Particularly when generating counterfactual candidates, it is easy to end up with out-of-distribution samples!

Combining uncertainty quantification & other explanations!

[5] Teufel et al. *Improving counterfactual truthfulness [...] through uncertainty quantification*. XAI'25 [link](#)

Coming back to counterfactuals



Particularly when generating counterfactual candidates, it is easy to end up with out-of-distribution samples!

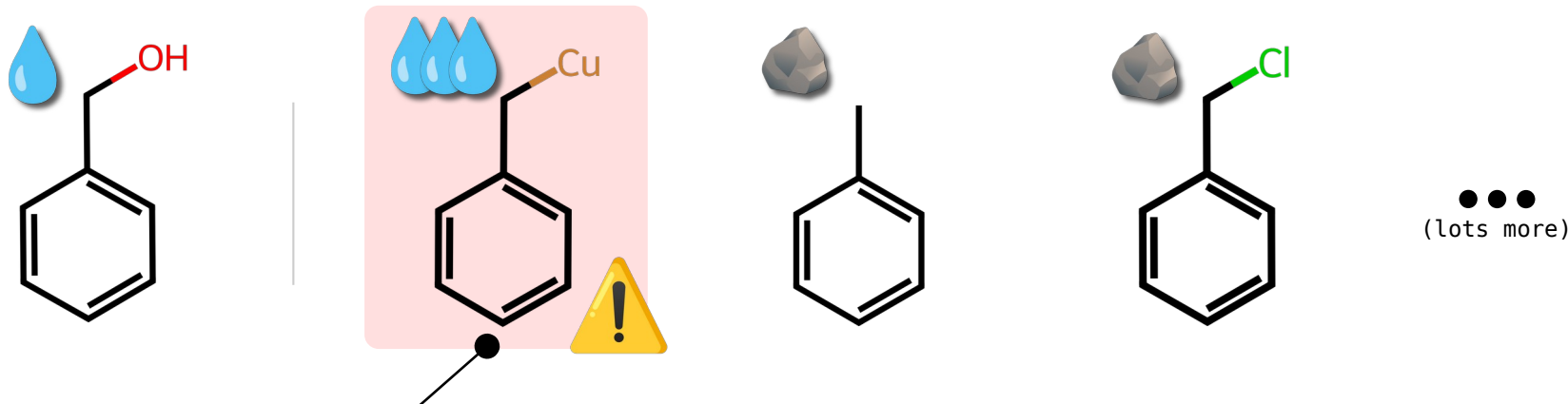


As an additional objective In the counterfactual optimization we can minimize the uncertainty of the candidates

Combining uncertainty quantification & other explanations!

[5] Teufel et al. *Improving counterfactual truthfulness [...] through uncertainty quantification*. XAI'25 [🔗](#)

Coming back to counterfactuals



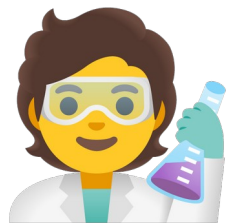
Particularly when generating counterfactual candidates, it is easy to end up with out-of-distribution samples!



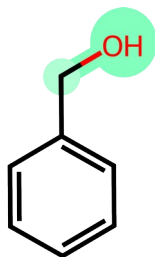
As an additional objective In the counterfactual optimization we can minimize the uncertainty of the candidates

$$\tilde{\mathbf{x}} = \arg \min_{\mathbf{x}'} (d(\mathbf{x}, \mathbf{x}') - \lambda_1 \cdot |y - y'| + \lambda_2 \cdot \sigma')$$

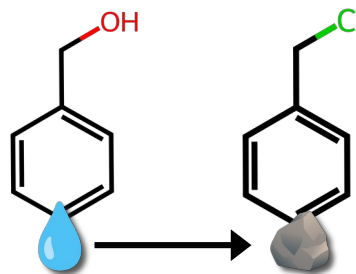
Can XAI help us understand our models? Our data?



Can XAI help us understand our models? Our data?

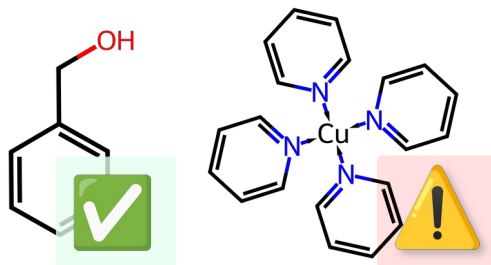
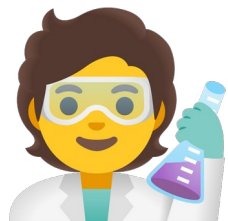


Attributional Explanations



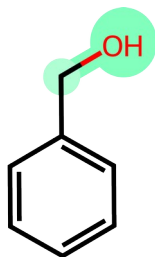
Counterfactuals

Now I understand!

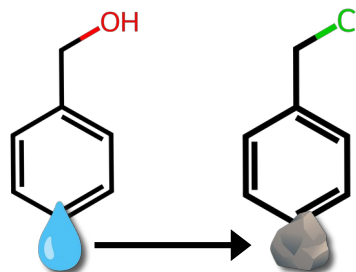


Uncertainty Quantification

Can XAI help us understand our models? Our data?

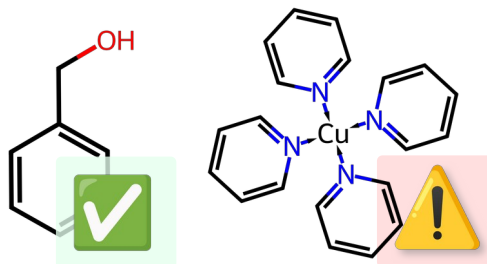
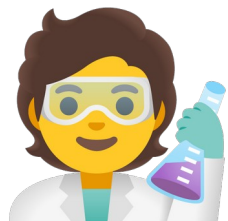


Attributional Explanations



Counterfactuals

Now I understand!



Uncertainty Quantification

Don't trust blindly!



Pitfalls

- Out-of-distribution effects (e.g. Counterfactuals)
- Explainer hyperparameter sensitivity
- Dataset biases
- ...

Thank you for your attention!